

**CSE4067**

# **Introduction to Blockchain Programming**

**Goshgar Can Ismayilov**

[goshgar.ismayilov@boun.edu.tr](mailto:goshgar.ismayilov@boun.edu.tr)

<https://www.linkedin.com/in/gismayilov/>

03.12.2024

# Consensus Models

Make multiple nodes agree on the same thing, e.g., which transaction to accept.

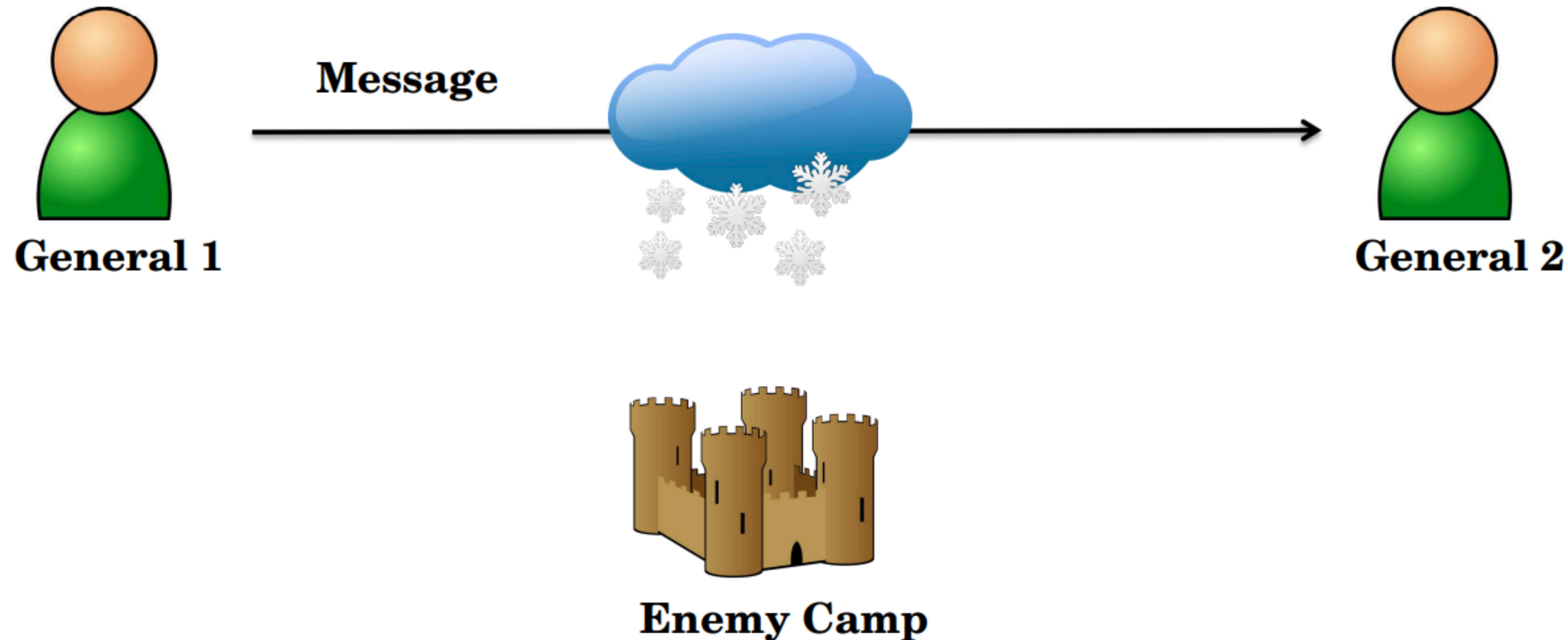
## **Why is this hard?**

- Nodes can be faulty or malicious\*
- Network delays might drop or reorder messages

## **Why is this even harder in the public blockchain setting?**

- Large scale network
- (Usually) not all participants are known
- Nodes have different hardware and operating systems

# The Two (Wisconsin) General's Problem

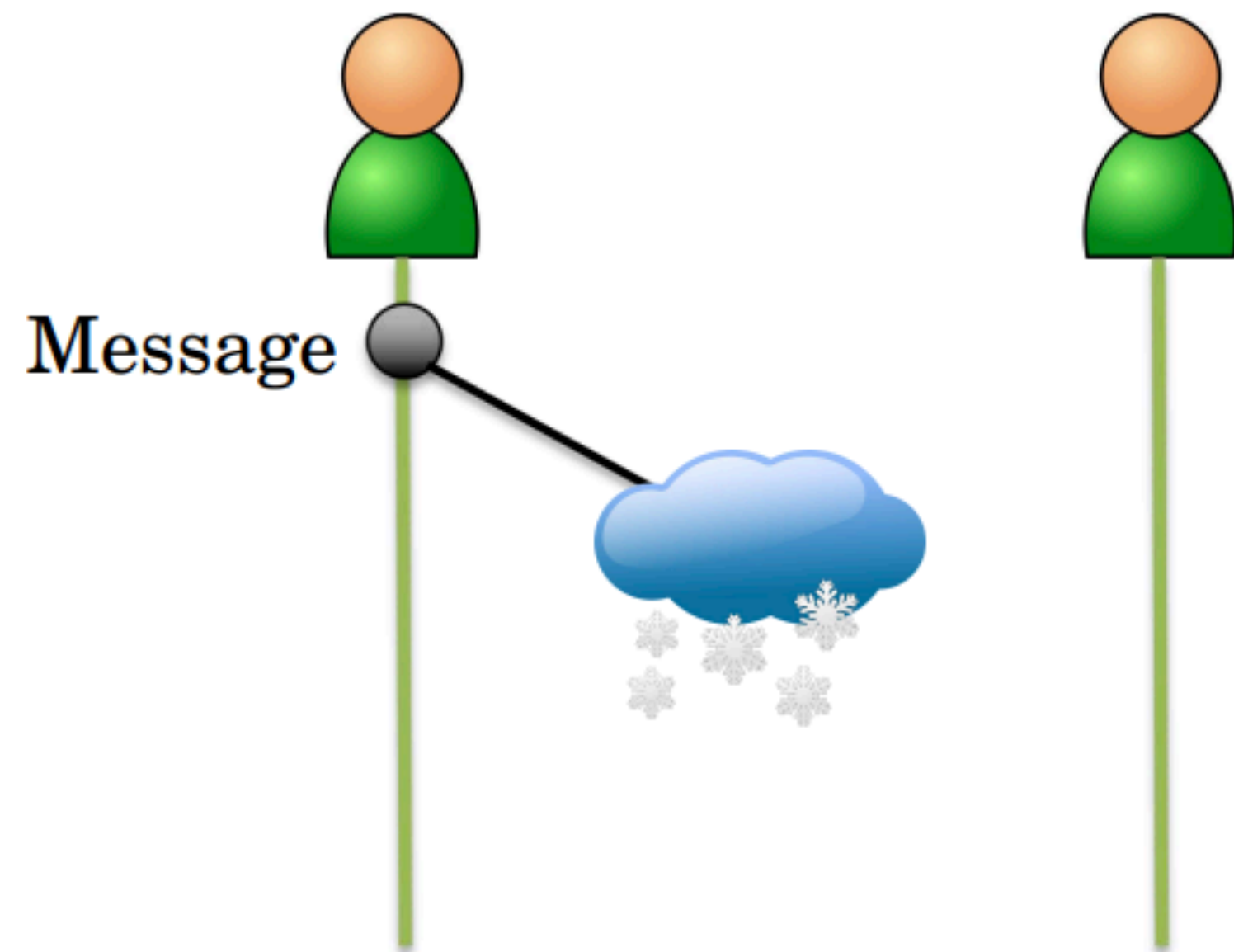


## Example to illustrate the challenges in achieving consensus

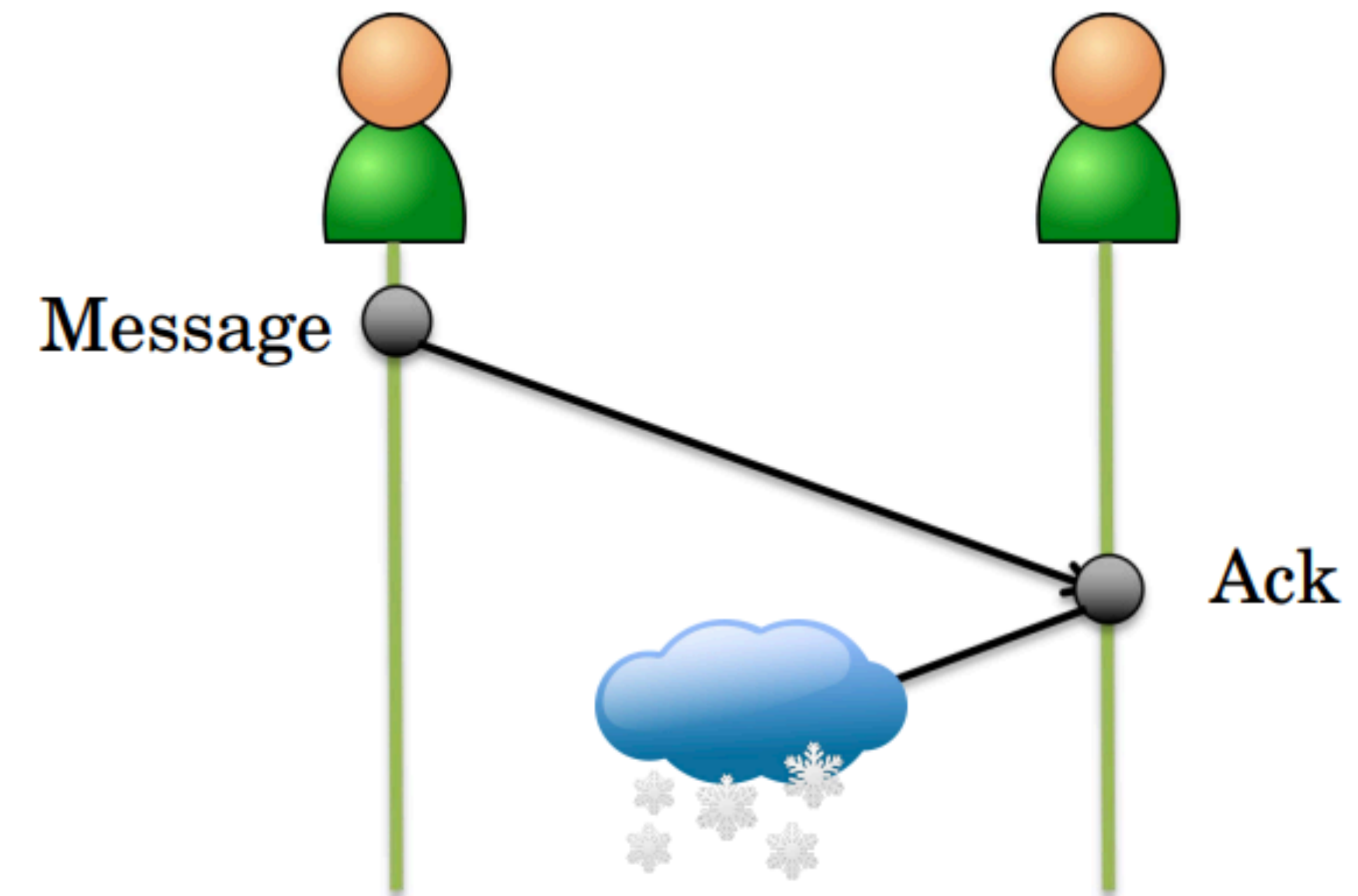
- Two generals have agreed to attack an enemy's camp
- The attack only succeeds if they attack at the same time
- They need to send a messenger through a snowstorm to agree on a time
  - There is a possibility the messenger dies in the storm and the message is lost

# The Two (Wisconsin) General's Problem

**Case 1: Message gets lost**



**Case 1: Acknowledgement gets lost**

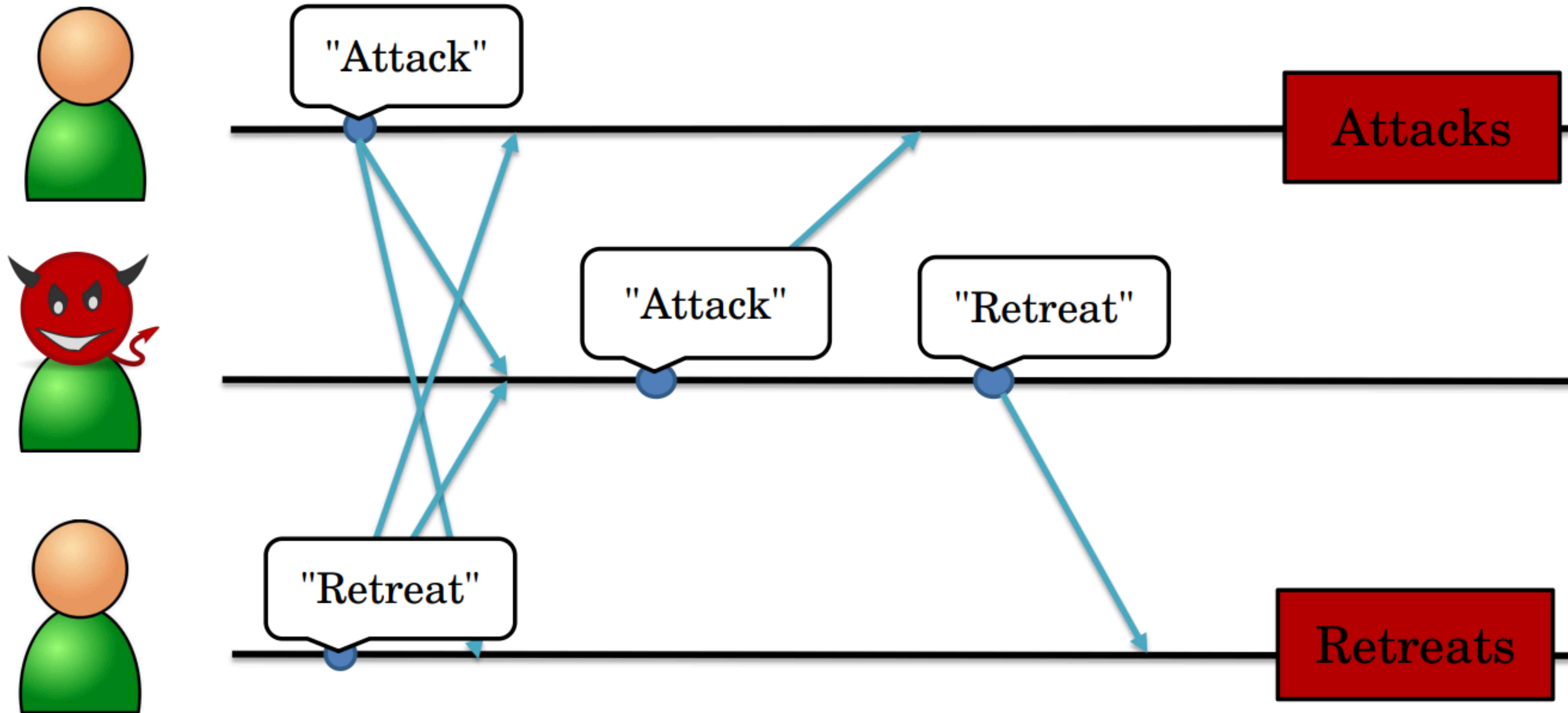


- To reach agreement, we need to know whether a message has arrived
- But the acknowledgement can also get lost

The problem, in this form, is unsolvable:

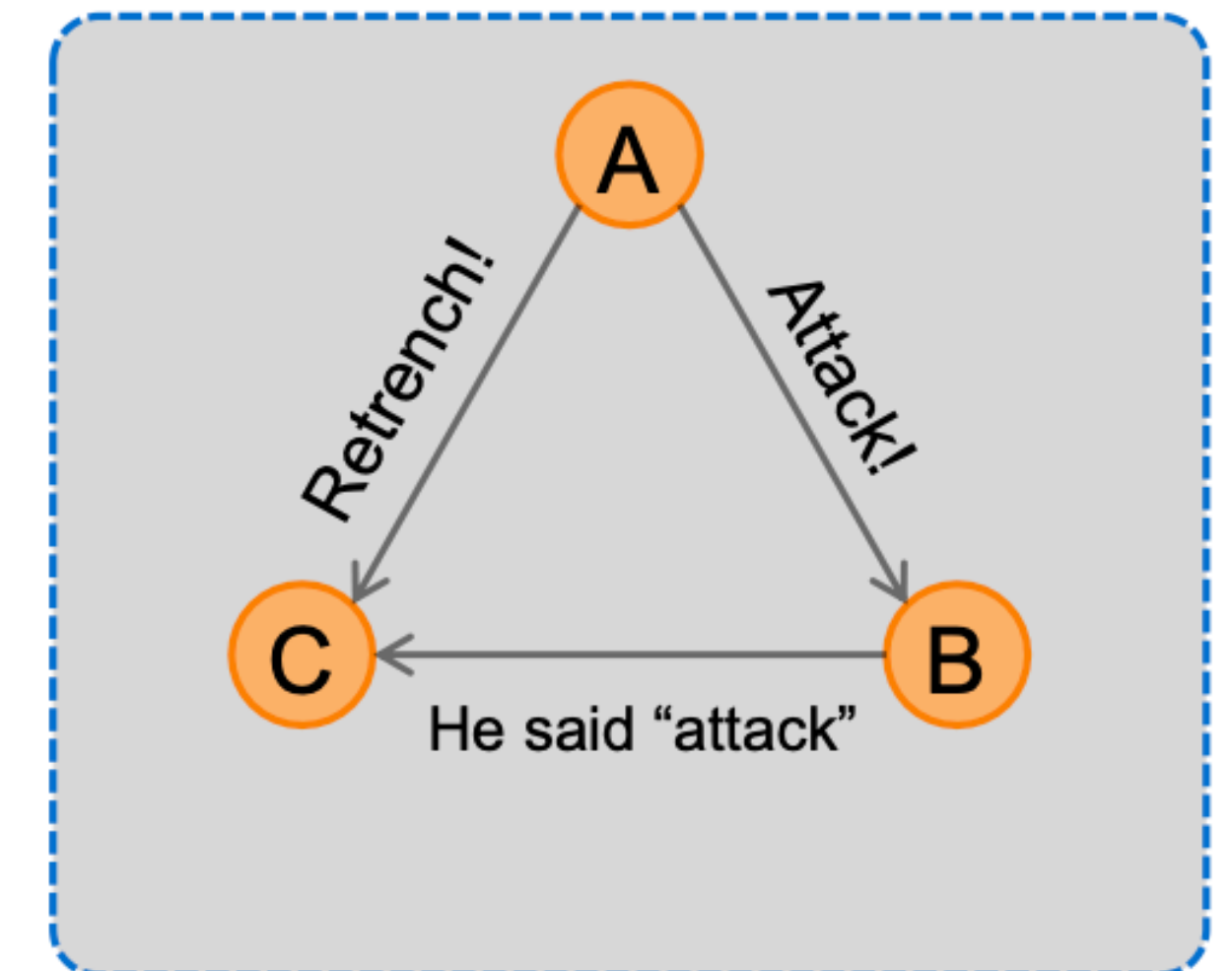
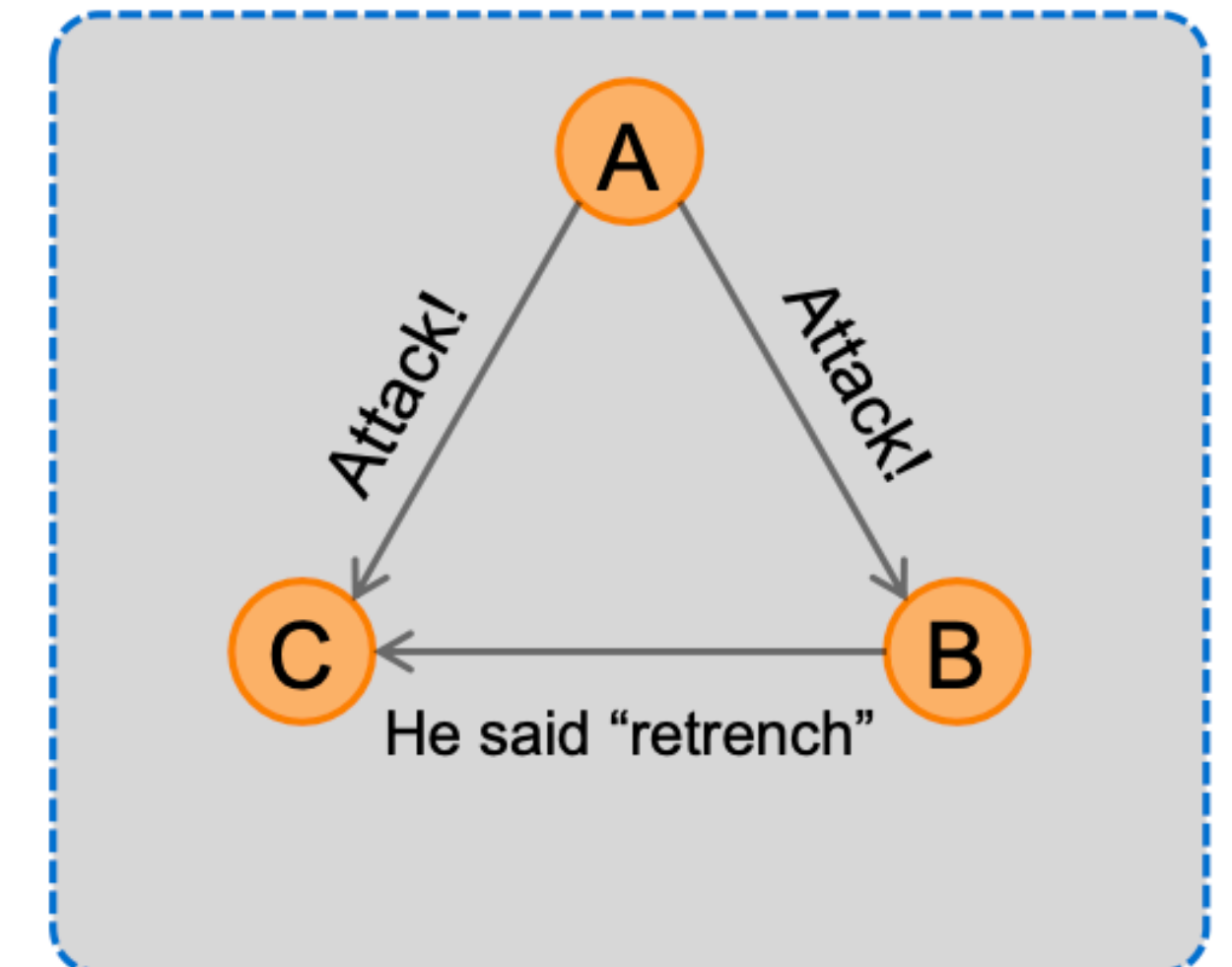
- We need *additional assumptions* about communication reliability to solve this

# Byzantine General's Problem



- Generals need to agree on whether or not to attack
- A malicious minority can cause the honest generals to adopt a bad decision

Three generals



C does not know what to agree on.

# Nakamoto Consensus

- Introduced with the Bitcoin paper in 2009
  - Named after the inventor's pseudonym

## Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto  
satoshin@gmx.com  
www.bitcoin.org

### A new class of consensus protocols

- First permissionless/public protocol
- Behaves in a probabilistic fashion
- Works well with large-scale networks

**Abstract.** A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

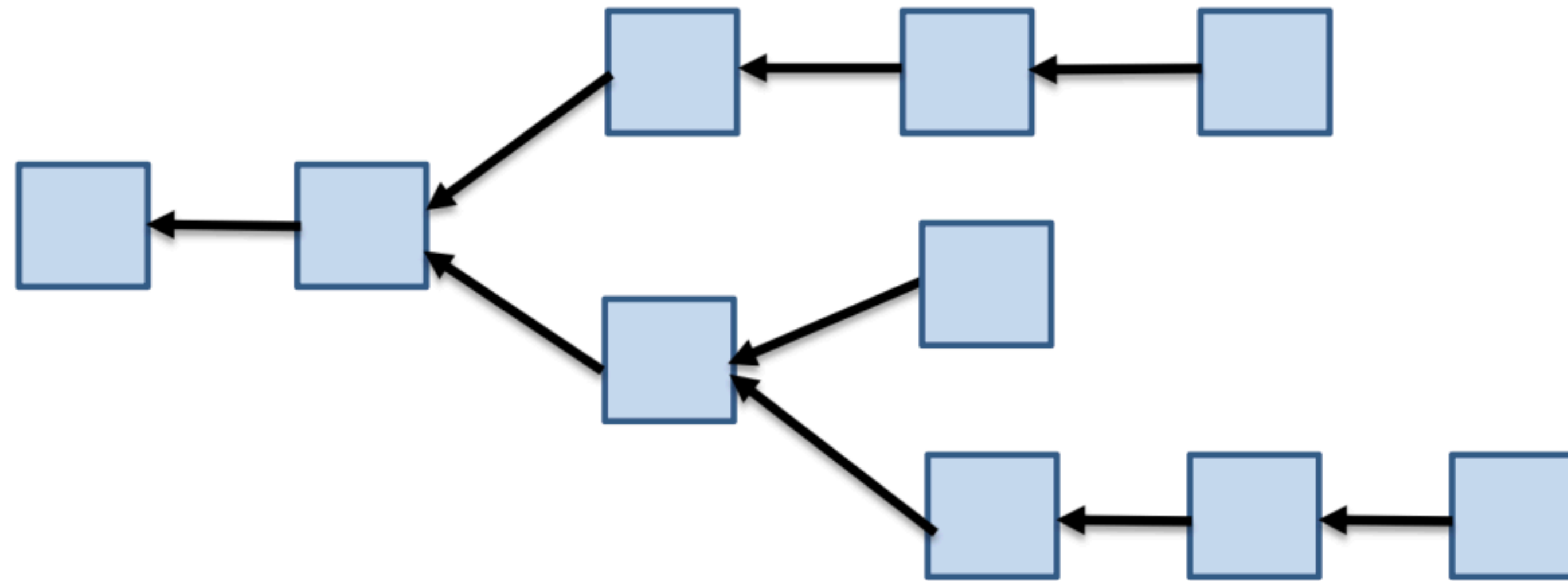
### 1. Introduction

Commerce on the Internet has come to rely almost exclusively on financial institutions serving as trusted third parties to process electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust based model. Completely non-reversible transactions are not really possible, since financial institutions cannot avoid mediating disputes. The cost of mediation increases transaction costs, limiting the minimum practical transaction size and cutting off the possibility for small casual transactions, and there is a broader cost in the loss of ability to make non-reversible payments for non-reversible services. With the possibility of reversal, the need for trust spreads. Merchants must be wary of their customers, hassling them for more information than they would otherwise need. A certain percentage of fraud is accepted as unavoidable. These costs and payment uncertainties can be avoided in person by using physical currency, but no mechanism exists to make payments

# Consensus Models

- Existing protocols are *deterministic*
  - We know which blocks/transactions have been accepted with full certainty
- Existing protocols are permissioned/*private*
  - All involved nodes are known
  - Adding or removing a node requires reconfiguration
- Most protocols, e.g., Paxos or PBFT, rely on a *leader*
  - One node is elected and in charge of generating blocks
  - Detecting leader failure and electing a new leader is tricky

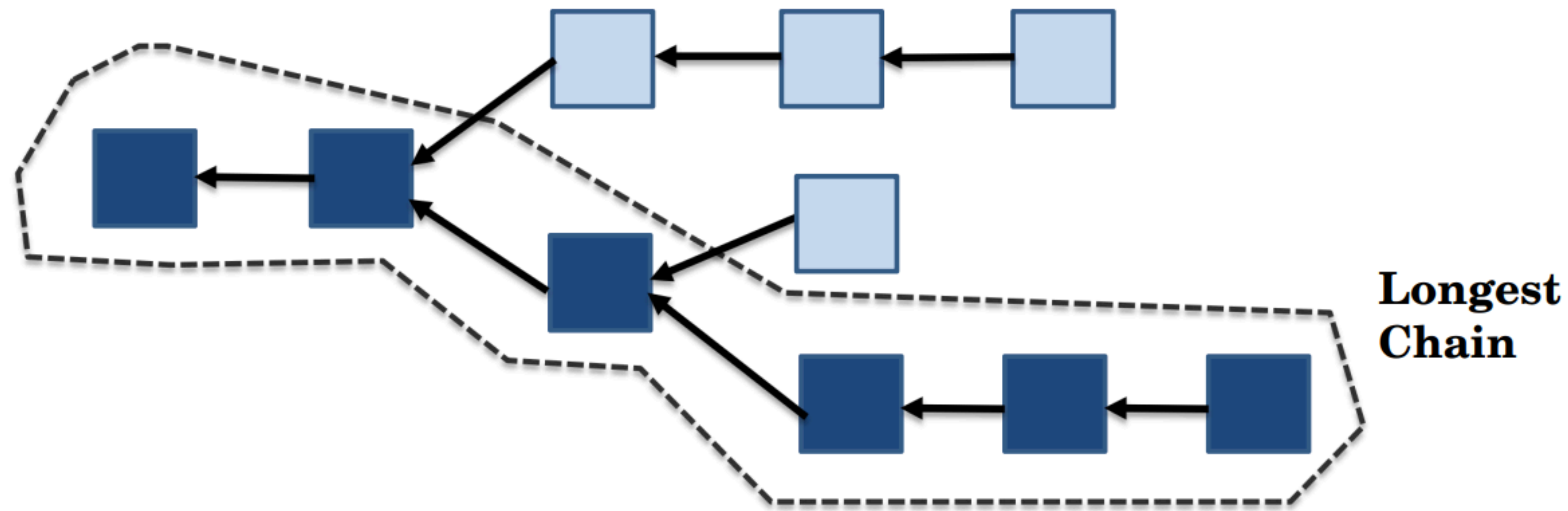
# Consensus Models



**Purpose 1:** Store transaction data and determine transaction ordering

**Purpose 2:** Track agreement on which transactions are accepted

# Nakamoto Consensus



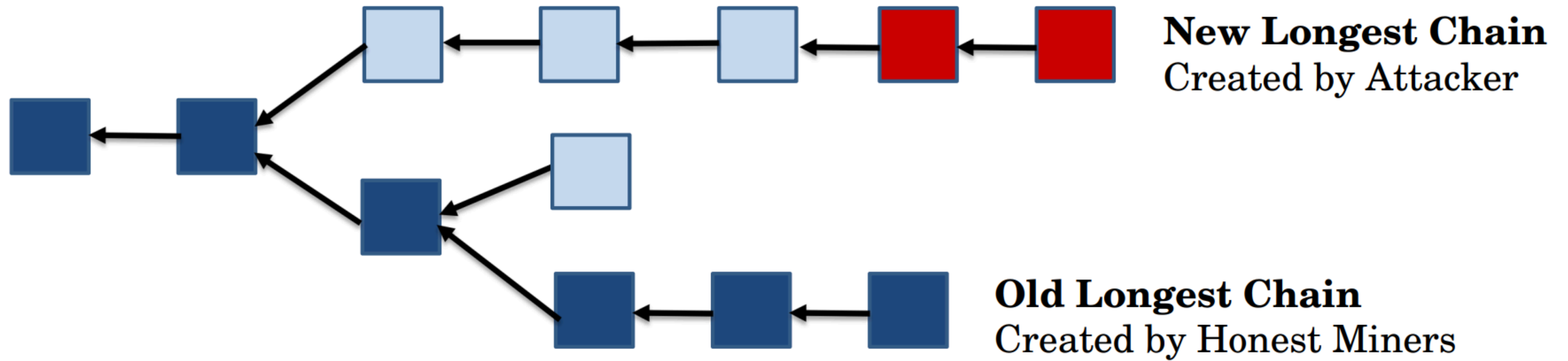
### Component 1: (Pseudo-)random Block Generation

- No pre-determined entity generates blocks, but virtually anyone can
- Multiple blocks can be created at the same time

## Component 2: Longest Chain Rule

- Correct nodes will always extend the longest chain when creating a new block
- When there are multiple longest chains, pick one at random

# Nakamoto Consensus: Safety



Attackers might intentionally build a fork to *overtake the honest chain*

**Worst Case:** One strong coordinated attacker

**Assumption:** Attacker's mining power  $< 50\%$

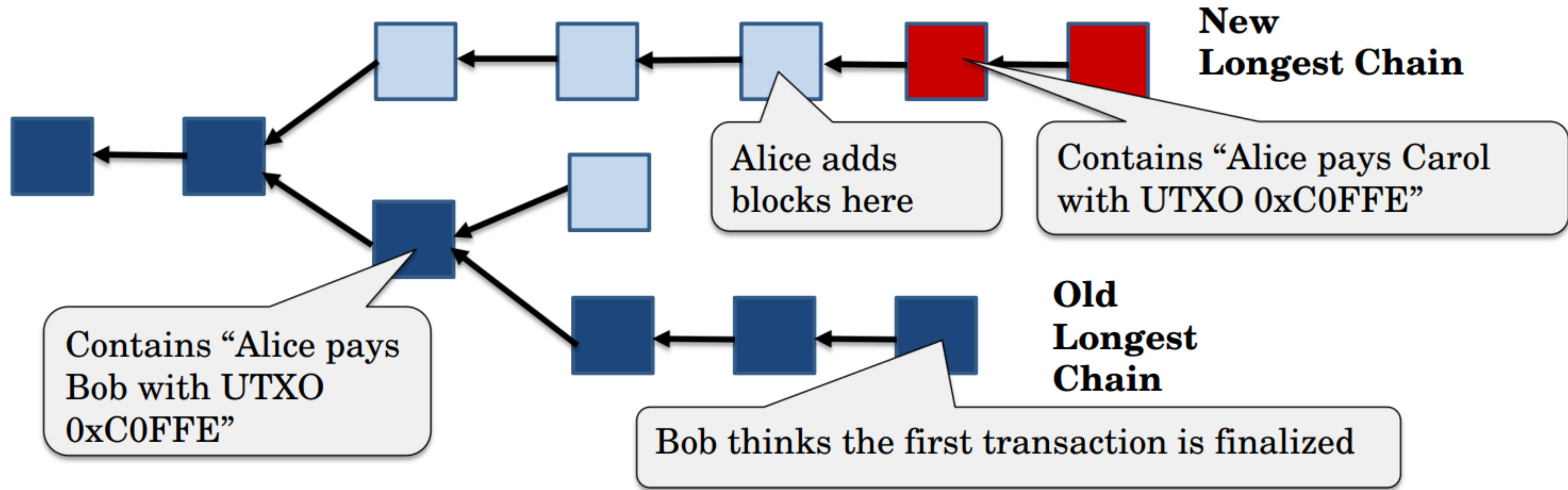
# Byzantine Failures in Nakamoto Consensus

Faulty nodes might

- Not extend the longest chain
- Send invalid blocks
- Create empty blocks
- Delay network messages

**What problems could this cause?**

# Double Spend Attack



## Goal

- When the network assumes a transaction is finalized, create some longer chain that reverts the transaction

# Nakamoto Consensus: Convergence

## Assumption 1: Synchronous network

- There exists some fixed time bound in which messages, e.g., blocks, will be delivered
- In Bitcoin the bound is usually assumed to be five minutes

## Assumption 2: Faulty nodes control a minority of the mining power

- Honest nodes create more blocks on average

At some point it is *virtually impossible* for the faulty chain to overtake the honest chain

|         |               |
|---------|---------------|
| $q=0.3$ |               |
| $z=0$   | $P=1.0000000$ |
| $z=5$   | $P=0.1773523$ |
| $z=10$  | $P=0.0416605$ |
| $z=15$  | $P=0.0101008$ |
| $z=20$  | $P=0.0024804$ |
| $z=25$  | $P=0.0006132$ |
| $z=30$  | $P=0.0001522$ |
| $z=35$  | $P=0.0000379$ |
| $z=40$  | $P=0.0000095$ |
| $z=45$  | $P=0.0000024$ |
| $z=50$  | $P=0.0000006$ |

Solving for  $P$  less than 0.1%...

|             |         |
|-------------|---------|
| $P < 0.001$ |         |
| $q=0.10$    | $z=5$   |
| $q=0.15$    | $z=8$   |
| $q=0.20$    | $z=11$  |
| $q=0.25$    | $z=15$  |
| $q=0.30$    | $z=24$  |
| $q=0.35$    | $z=41$  |
| $q=0.40$    | $z=89$  |
| $q=0.45$    | $z=340$ |

**"Figure" from the paper:**

$q$  is the mining power of the attacker  
 $z$  is the length of the "honest" chain

# Sybil Attack

A malicious entity might try to configure many nodes to take over the network

- These are called *Sybils*
- We need some mechanism to detect or weaken Sybils

In permissioned/private protocols, the members (set of nodes) are pre-defined

- New nodes can only be added with a reconfiguration

In permissionless/public protocols, we need a dedicated Sybil-resistance mechanisms

- e.g., Proof-of-Work (today) or Proof-of-Stake (future lecture)

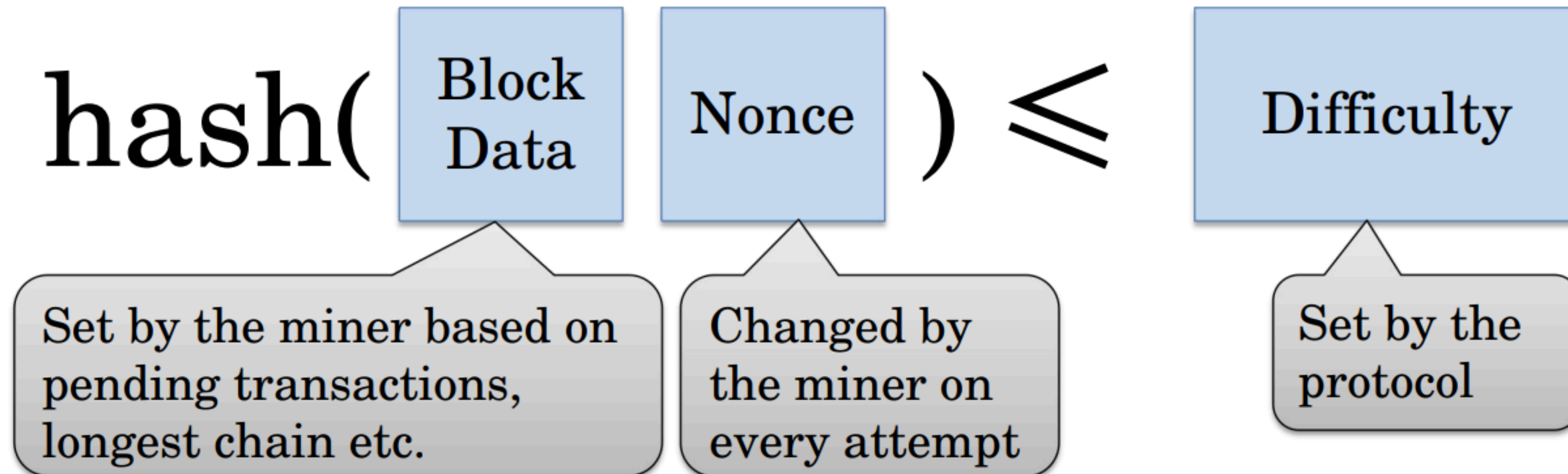
# Proof-of-Work (PoW)

**Goal:** Tie likelihood of generating a block to processing power

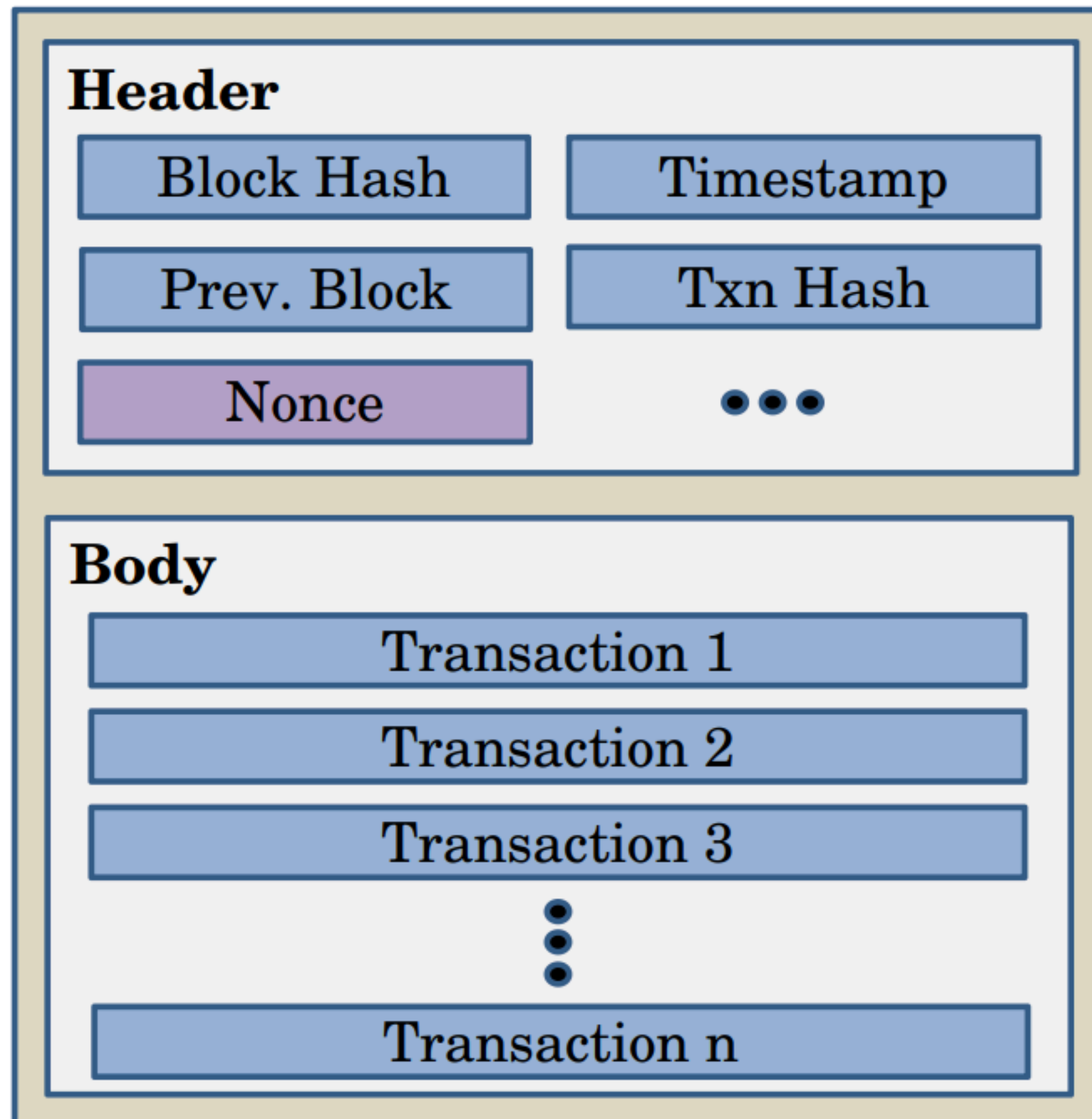
- Each node only has some finite amount of hardware

**Approach:** Create a very hard-to-solve task (the “crypto puzzle”)

- Random tries are needed to find the solution
- We might need many attempts to solve it



# Proof-of-Work (PoW)



- Miners pick a random *nonce* value
- For a block to be valid, the block header's hash needs to be below some difficulty value
- Chance of mining a valid block at any point in time is *independent* of time already spent mining

# Proof-of-Work (PoW): Dynamic Availability

So far:  $n$  nodes maintain the blockchain

This number may change over time:

- new users enter the system
- existing users leave

The change over time can be **dramatic**

The Bitcoin blockchain handles this, by adjusting the target (difficulty) of the Proof of Work algorithm

# Proof-of-Work (PoW): Difficulty

**Goal:** Ensure that a blocks are created at the same frequency

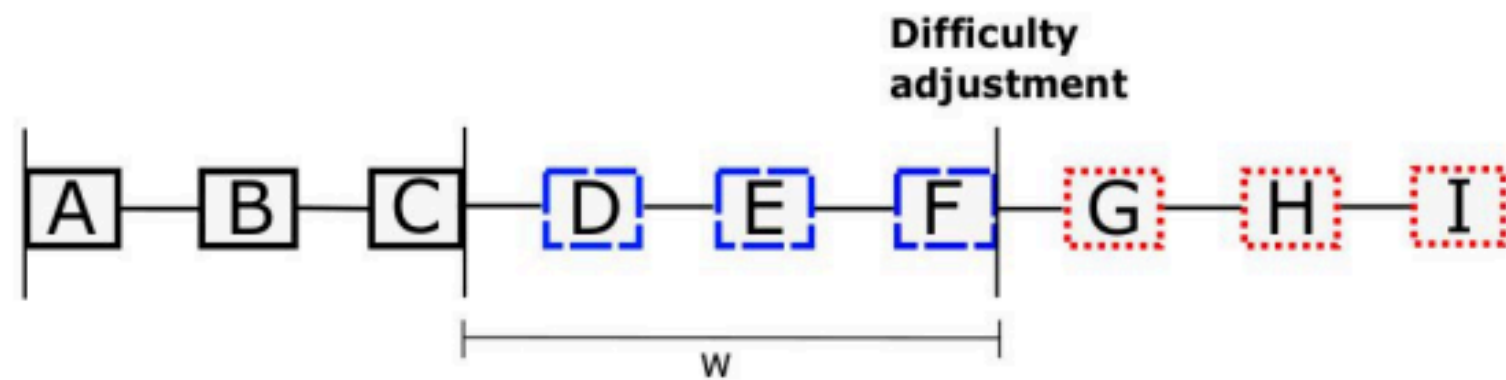
- e.g., every 10 Minutes in Bitcoin

**Challenge:** Mining power can change over time

- Miners can join or leave at any time
- Miners might start (or stop) mining if it is (not) economical to so
  - Depends on electricity, cryptocurrency, and hardware prices
- Miners can switch between networks (e.g., from Bitcoin to Dogecoin)

Adjust difficulty based on the *observed frequency vs. the expected frequency*

# Proof-of-Work (PoW): Difficulty



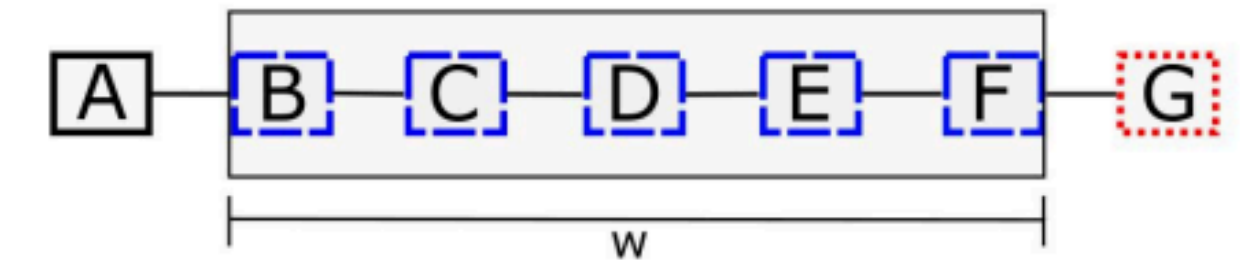
## Period-Based

- Every  $w$  blocks the difficulty will be adjusted
- For example, in Bitcoin every 2016 blocks (roughly 2 weeks) the difficulty is recalculated



## Incrementally Extrapolated

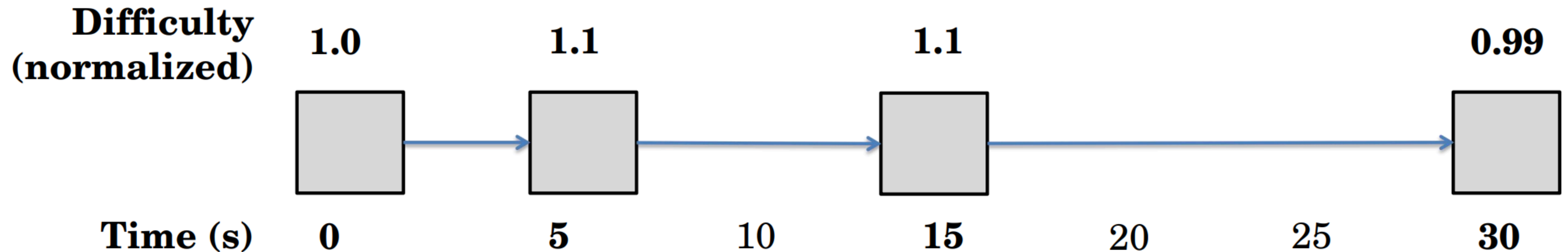
- Every block the difficulty will be adjusted slightly depending on how long it took to mine it
- Difficulty is only adjusted, not recalculated
- Used by Ethereum



## Sliding Window

- Every block the difficulty will be adjusted depending on how long it took to mine the last  $w$  blocks
- Used by Monero and Bitcoin Cash
- Different implementations have varying window sizes and mechanism to deal with outliers

# Proof-of-Work (PoW): Difficulty



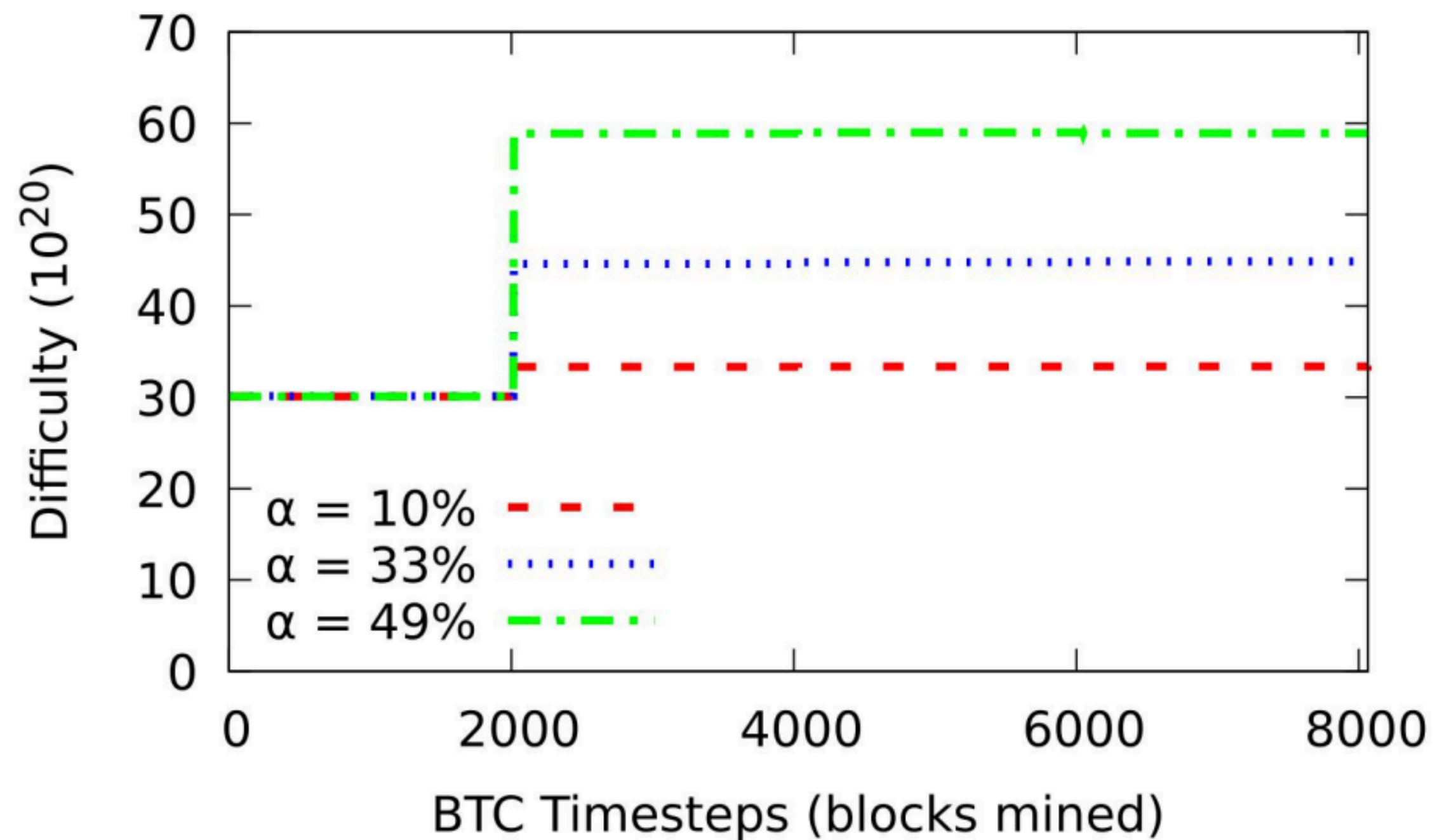
## Simplified Incremental Extrapolation

```
delta_t = block.time - block.get_parent().time
expected = 12 # Ideally it should take 12 seconds
if delta_t < expected-2: # Allow for some error
    difficulty *= 1.1 # Increase difficulty
elif delta_t > expected+2:
    difficulty *= 0.9 # Decrease difficulty
```

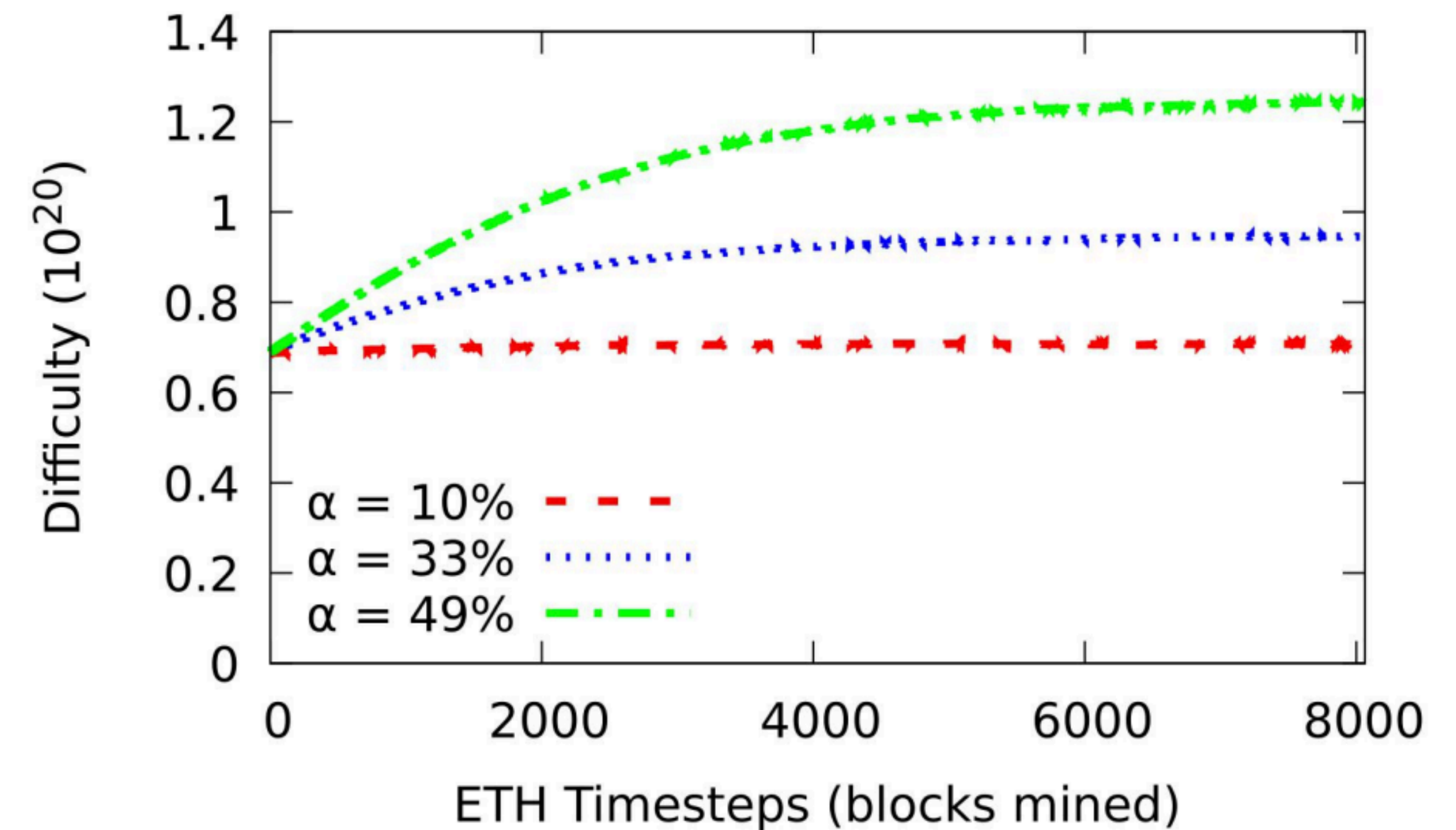
# Proof-of-Work (PoW): Difficulty

A new powerful miner joins the network (at  $T=0$ ). The new miner has  $\alpha$  of the total mining power.

How long does it take to adjust difficulty?



Bitcoin adjusts difficulty every 2016 blocks



Ethereum adjusts difficulty incrementally

# Proof-of-Work (PoW): Incentive

Majority of miners (or mining power) is “honest”

- Will follow protocol as long as it makes financial sense
- Mining is very costly

We need a mechanism to make mining profitable

- Block rewards and transaction fees are paid to the miner
- Rewards are only paid out if block is on the winning chain
  - Ensures convergence to a longest chain

Lower difficulty means, potentially, higher rewards for miners!

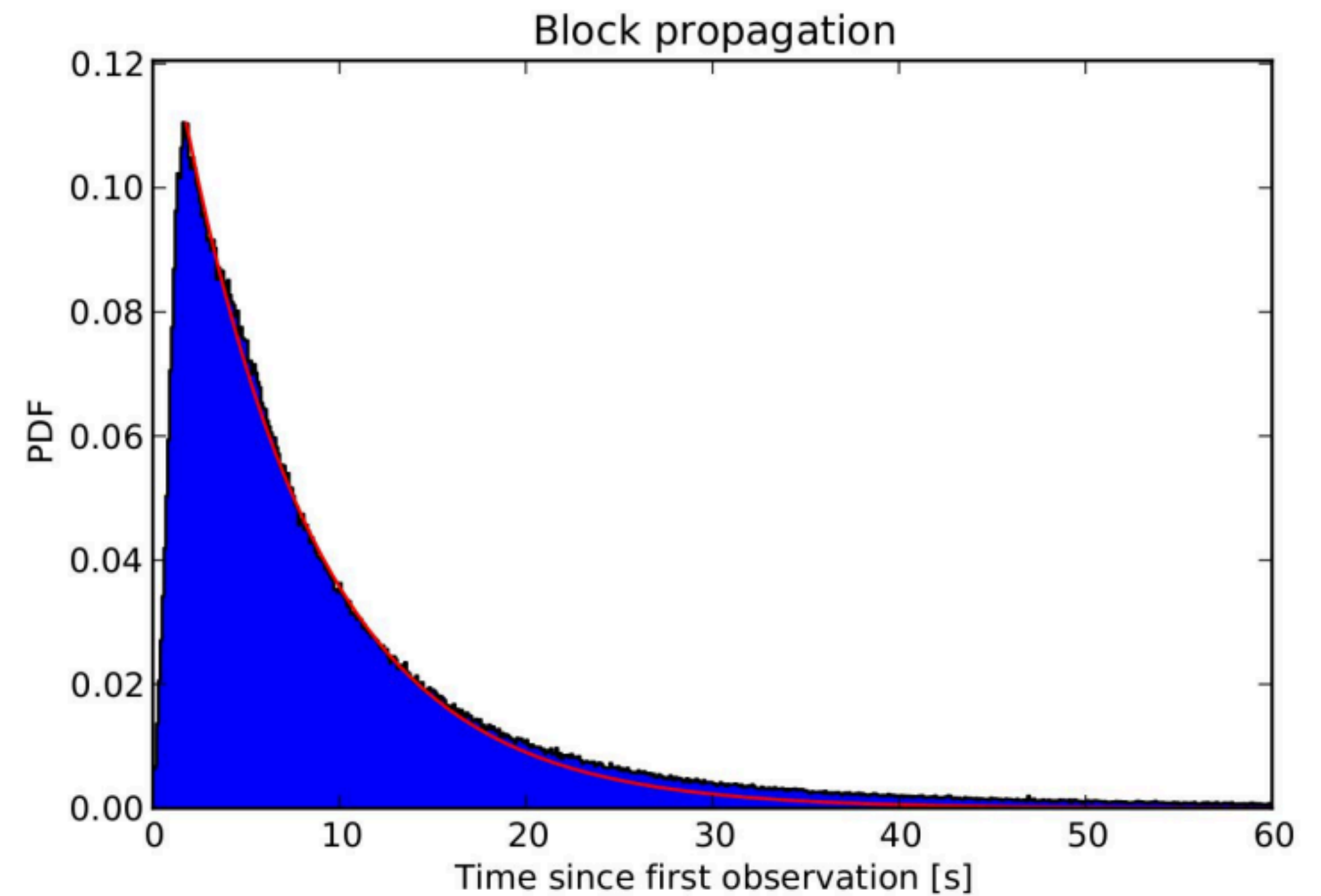
# Proof-of-Work (PoW)

## **"Information propagation in the Bitcoin network" (Decker and Wattenhofer, 2013)**

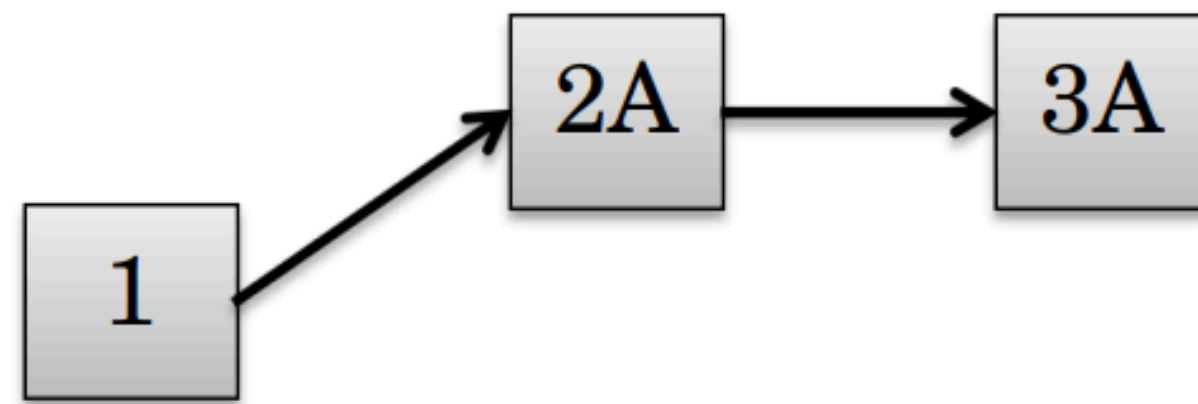
- 95% of nodes can be reached in <13seconds
- 50% of nodes are reached within 6 seconds
- Numbers might be slightly different today

## **Why does block propagation take so long?**

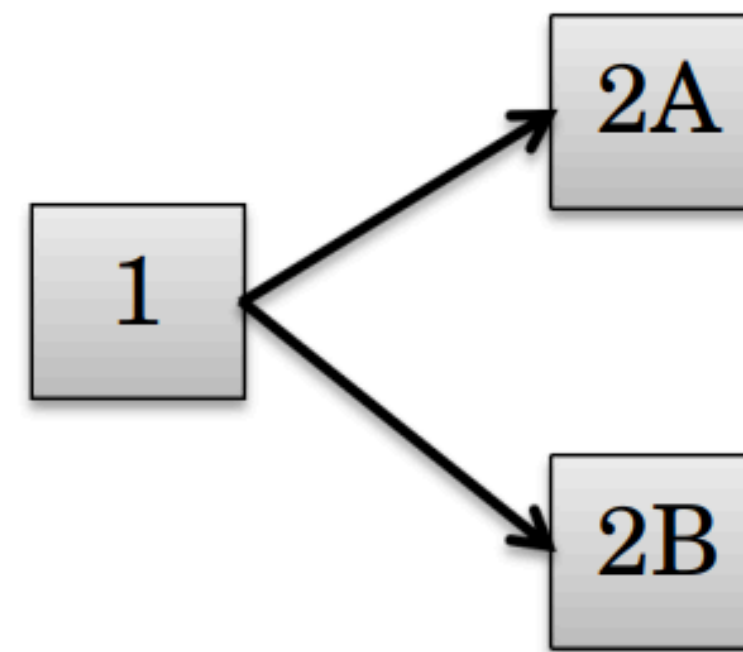
- Nodes verify/execute blocks before forwarding
- Gossip network introduces additional network hops



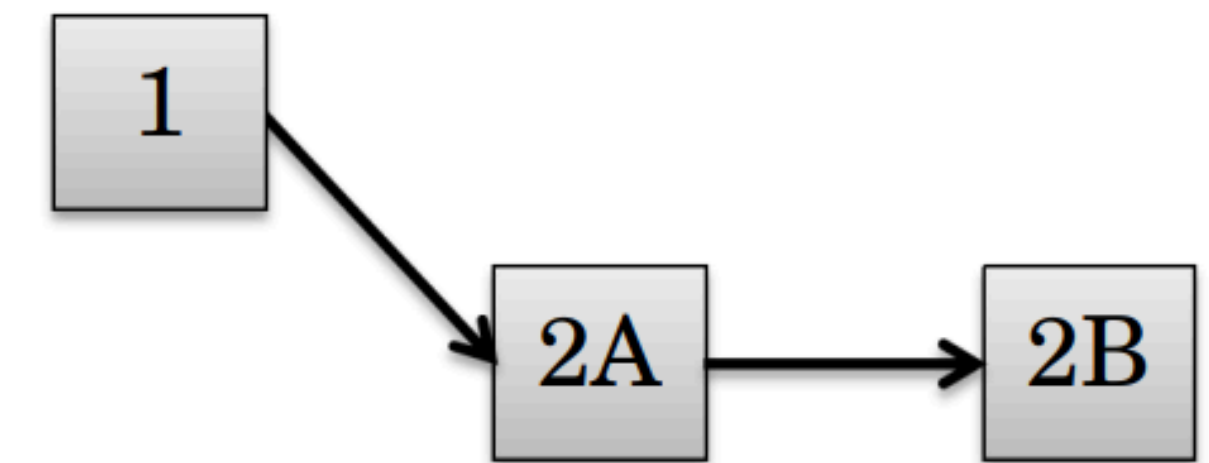
# Proof-of-Work (PoW): Information Asymmetry



**Node 1's View**



**Node 2's View**



**Node 3's View**

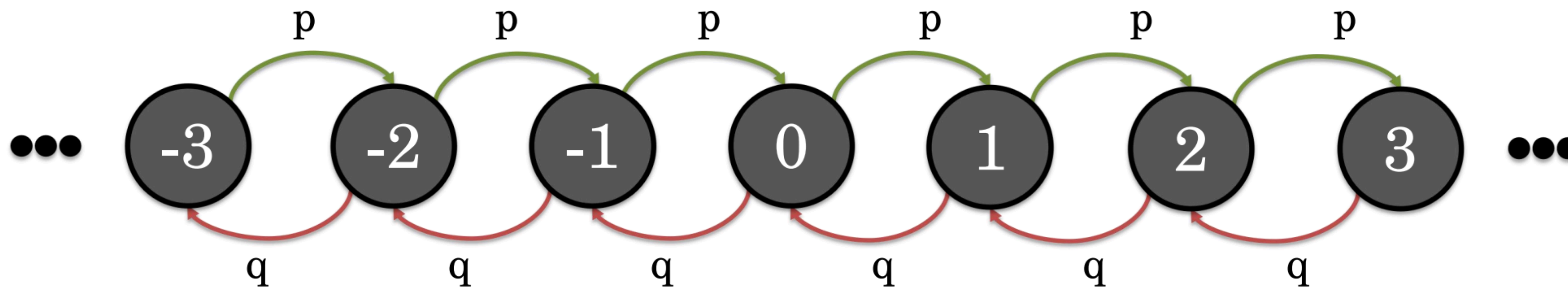
- Each node sees some subset of all blocks
- In Bitcoin and Ethereum 1.0 there is no certain way of knowing which blocks have been seen by a majority of nodes

## Why?

- Network failures and delays
- Attackers might not forward blocks

# Proof-of-Work (PoW)

## ADVANTAGE OF HONEST CHAIN



**Honest Mining Power:  $p$**

**Attacker's Mining Power:  $q = 1 - p$**

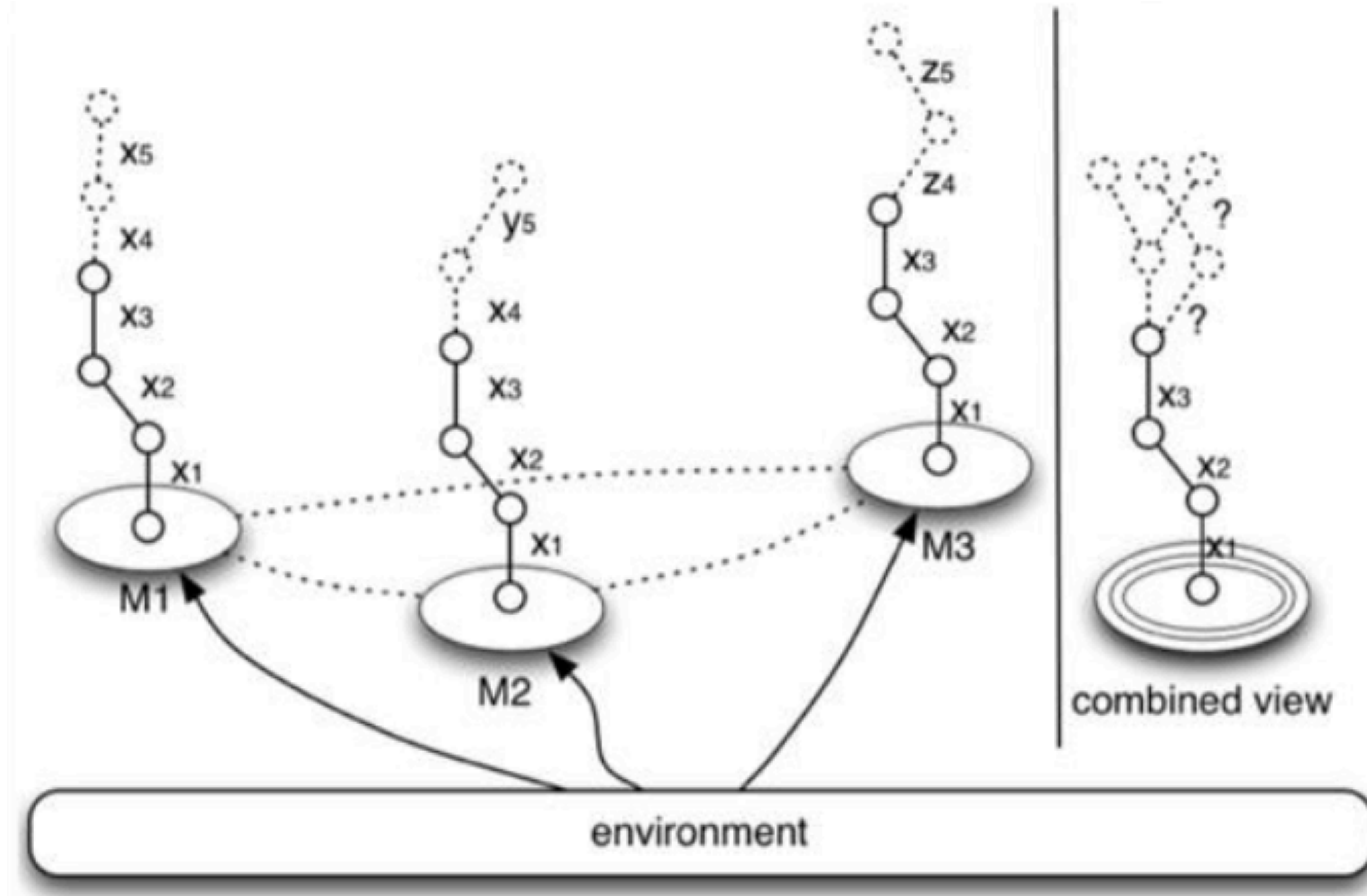
Chance the miner overtakes after  $z$  blocks  
**asymptotically approaches 0**

|         |               |
|---------|---------------|
| $q=0.3$ |               |
| $z=0$   | $P=1.0000000$ |
| $z=5$   | $P=0.1773523$ |
| $z=10$  | $P=0.0416605$ |
| $z=15$  | $P=0.0101008$ |
| $z=20$  | $P=0.0024804$ |
| $z=25$  | $P=0.0006132$ |
| $z=30$  | $P=0.0001522$ |
| $z=35$  | $P=0.0000379$ |
| $z=40$  | $P=0.0000095$ |
| $z=45$  | $P=0.0000024$ |
| $z=50$  | $P=0.0000006$ |

Solving for  $P$  less than 0.1%...

|             |         |
|-------------|---------|
| $P < 0.001$ |         |
| $q=0.10$    | $z=5$   |
| $q=0.15$    | $z=8$   |
| $q=0.20$    | $z=11$  |
| $q=0.25$    | $z=15$  |
| $q=0.30$    | $z=24$  |
| $q=0.35$    | $z=41$  |
| $q=0.40$    | $z=89$  |
| $q=0.45$    | $z=340$ |

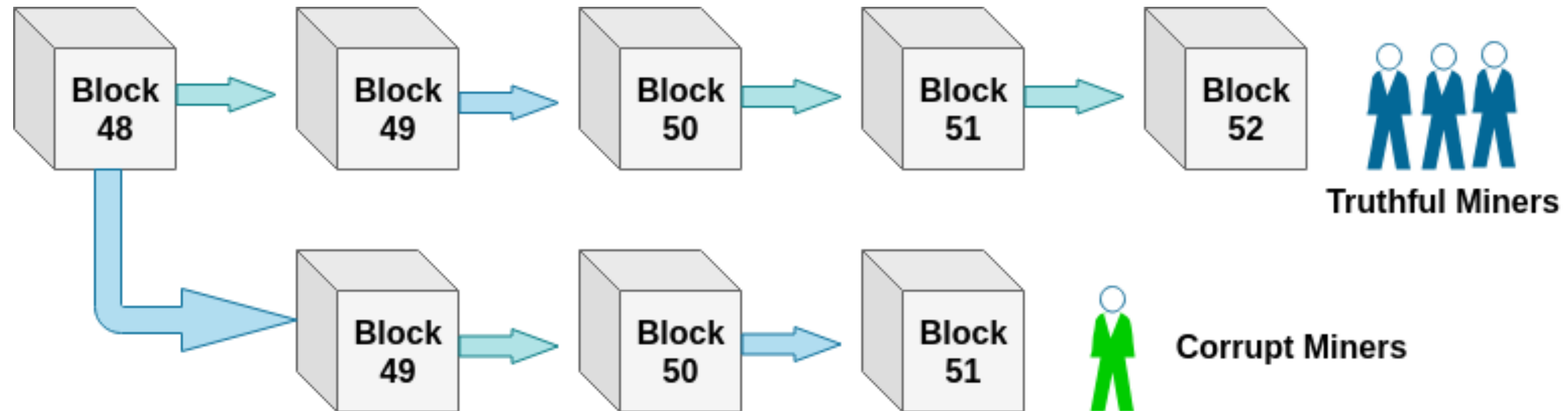
# Consensus Models: Common Prefix



# Consensus Models: Racing Attack

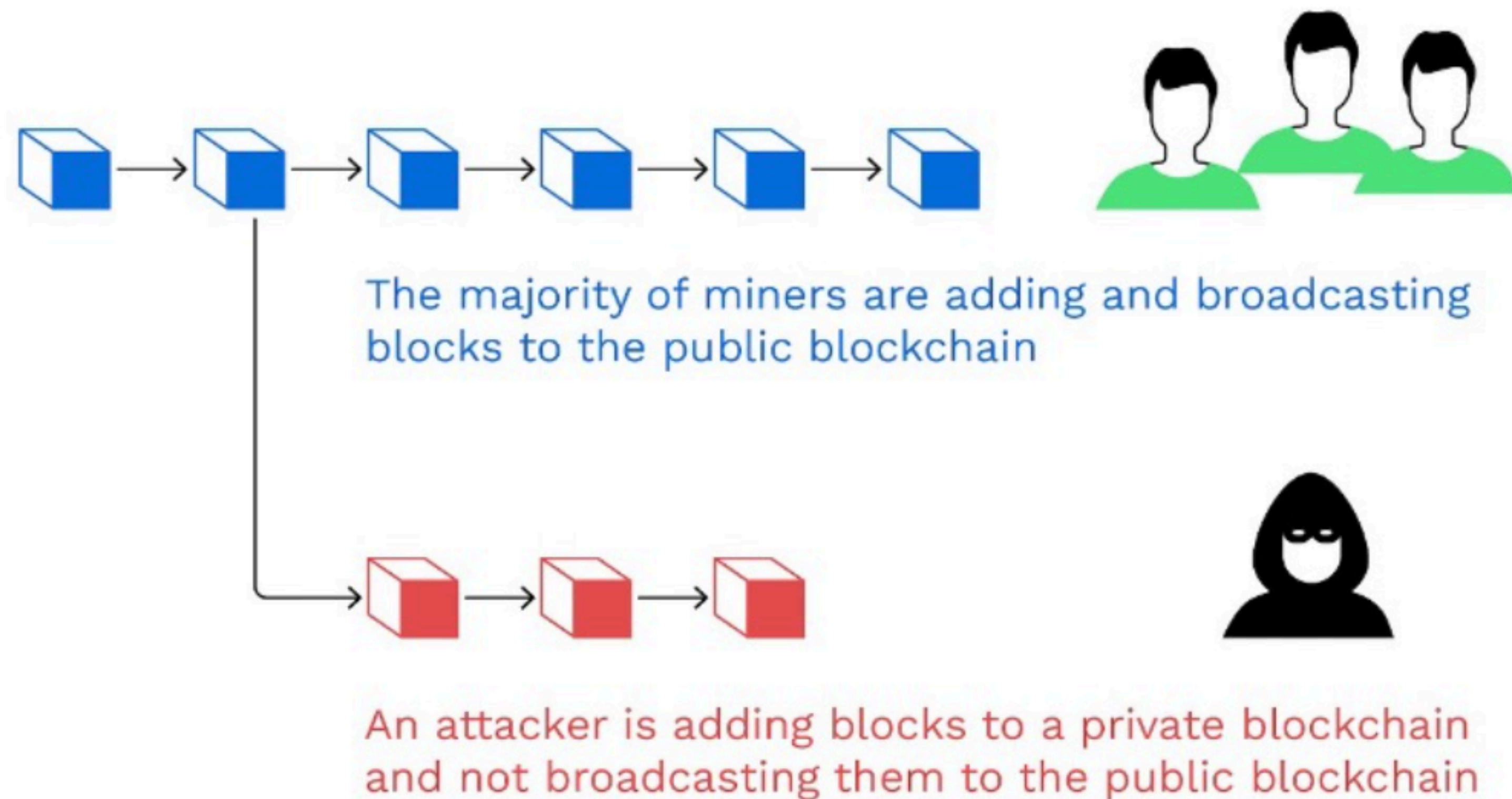
Attacker splits from the main chain and tries to overtake the “honest chain”

=> Common prefix breaks



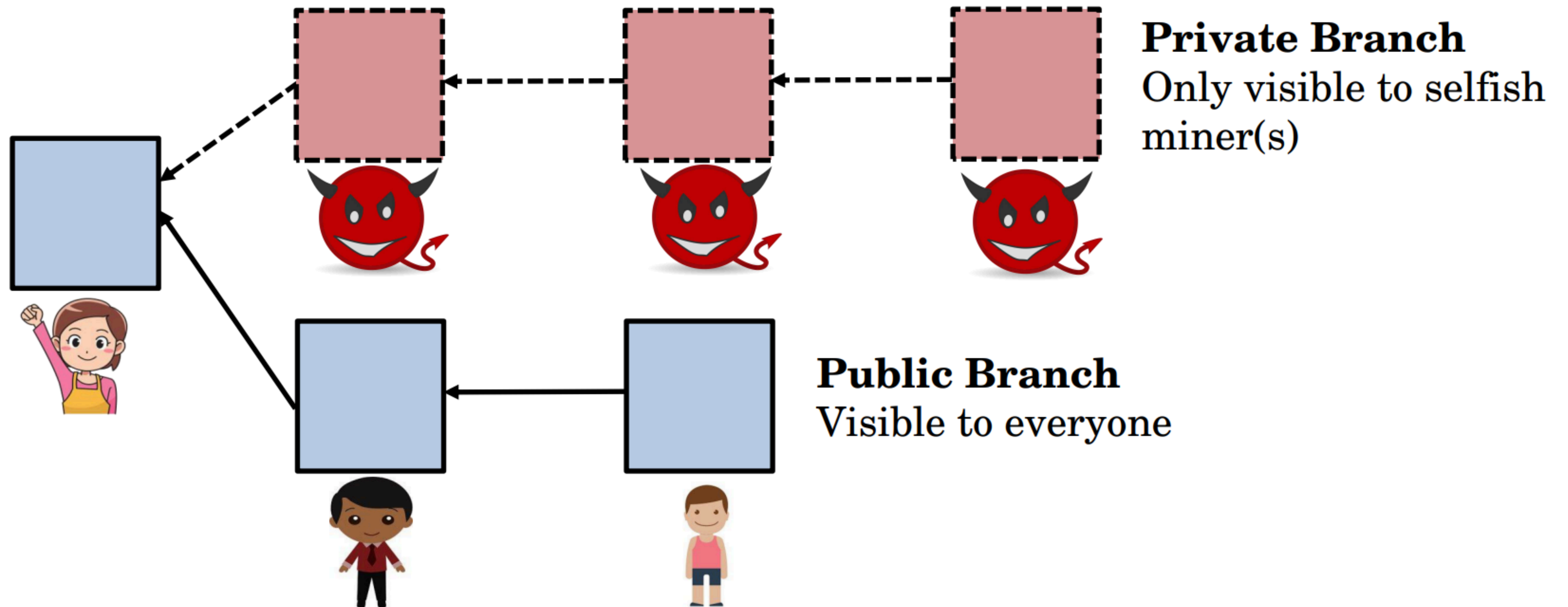
# Consensus Models: Withholding Attack

Attacker mines privately and releases their block at the same time an honest party releases its own block



# Consensus Models: Selfish Mining

**Idea:** Selectively hide blocks you mine to gain an advantage



# Consensus Models: Selfish Mining

**Goal 1:** Keep mined blocks hidden as long as possible

- Use "information asymmetry" to your advantage
- Other miners cannot mine on hidden blocks

**Goal 2:** Try to have two competing branches exist for as long as possible

- If there are multiple competing branches, honest mining power might be split between them

**Approach:** When honest miners publish a block, try to reveal one (or more) blocks from the private branch to "confuse" honest majority

# Selfish Mining: When to Publish Blocks

**When** selfish miner finds a block

**and** private branch is now longer than public branch

**and** private branch is now exactly two blocks

**then** publish entire private branch (*selfish miner wins*)

**When** honest miner finds a block **and** public and private branch now have the *same length*

**then** publish one block (*confuse honest miners*)

**When** honest miner finds a block **and** private branch is *exactly two blocks longer*

**then** publish entire private chain (*selfish miner wins*)

**When** honest miner finds a block **and** private branch is still *more than two blocks longer*

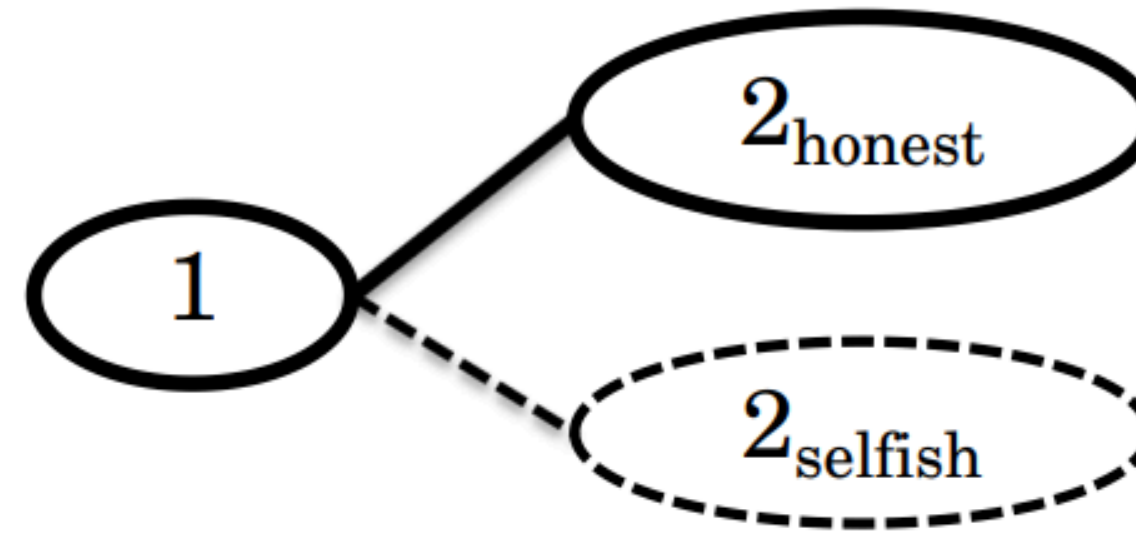
**then** publish one block (*confuse honest miners*)

**When** honest miner finds a block **and** *public branch is longer* than private branch

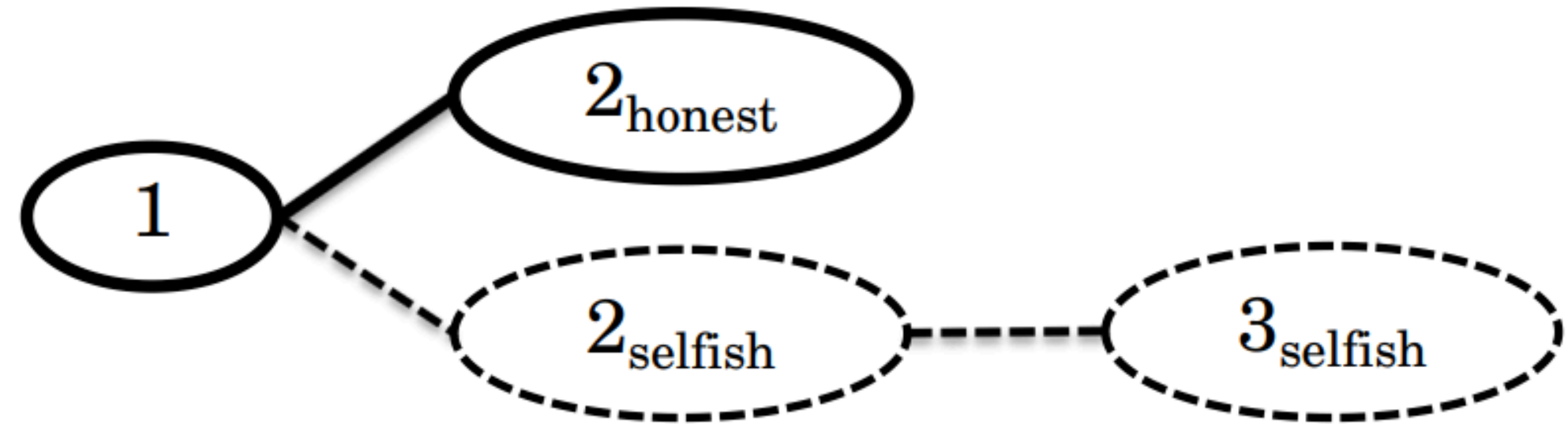
**then** switch to mining on the public branch (*honest miners win*)

# Selfish Mining: Example 1

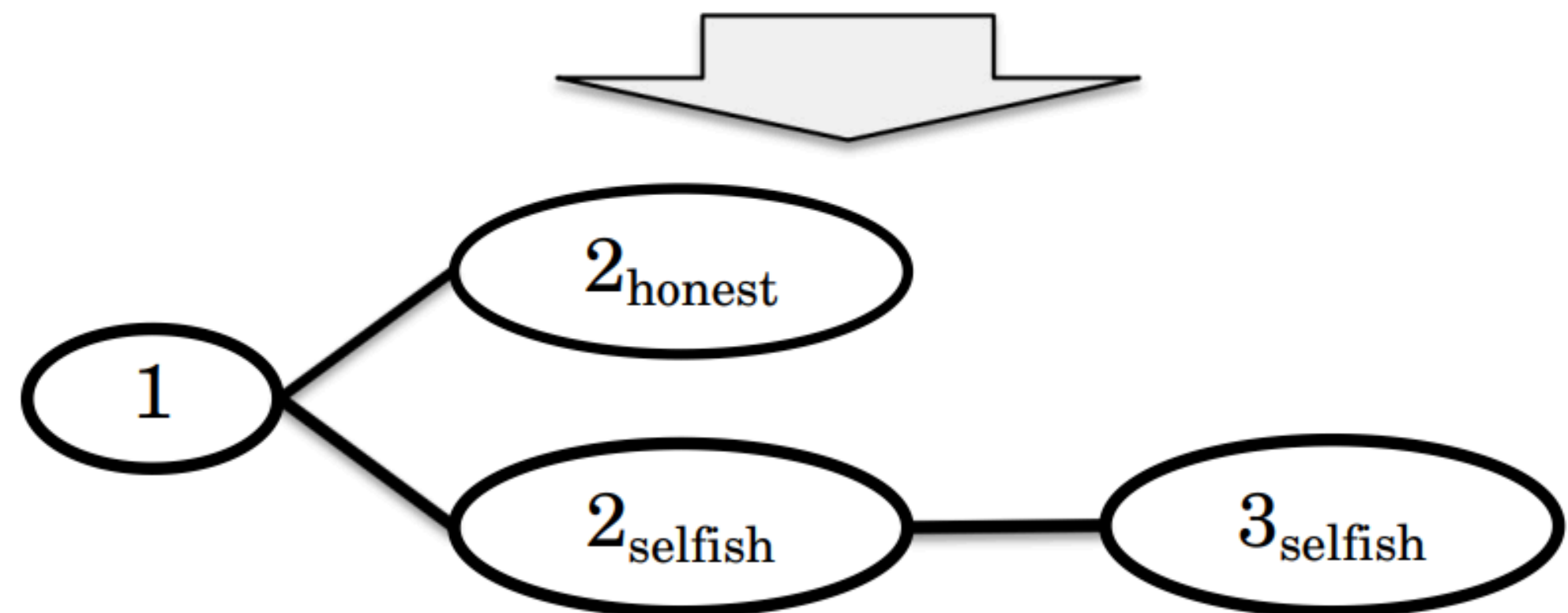
1) Private and public branch are both of length one



2) Selfish miner finds a new block

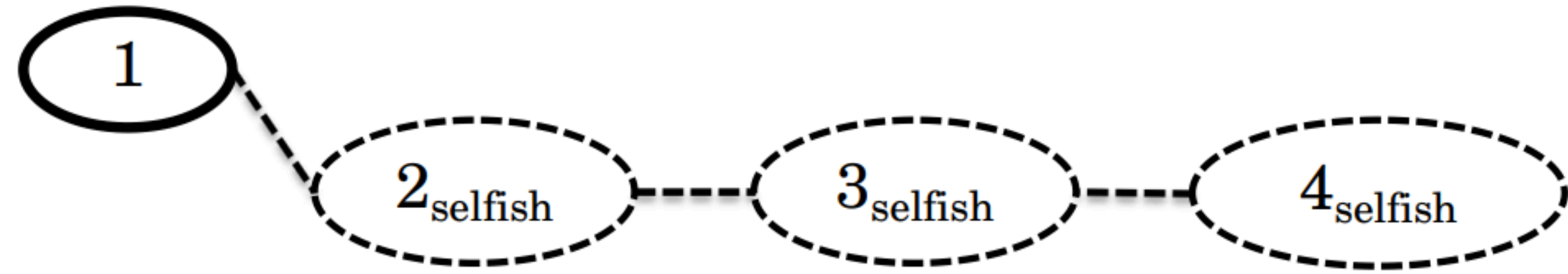


3) Selfish miner reveals both of its blocks and wins  
(2<sub>honest</sub> becomes orphaned)

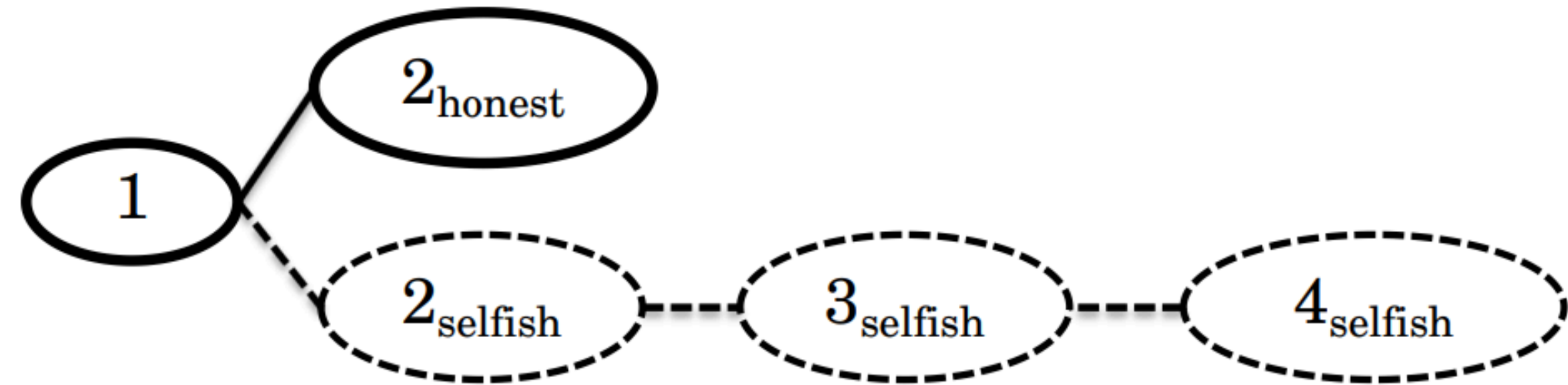


# Selfish Mining: Example 2

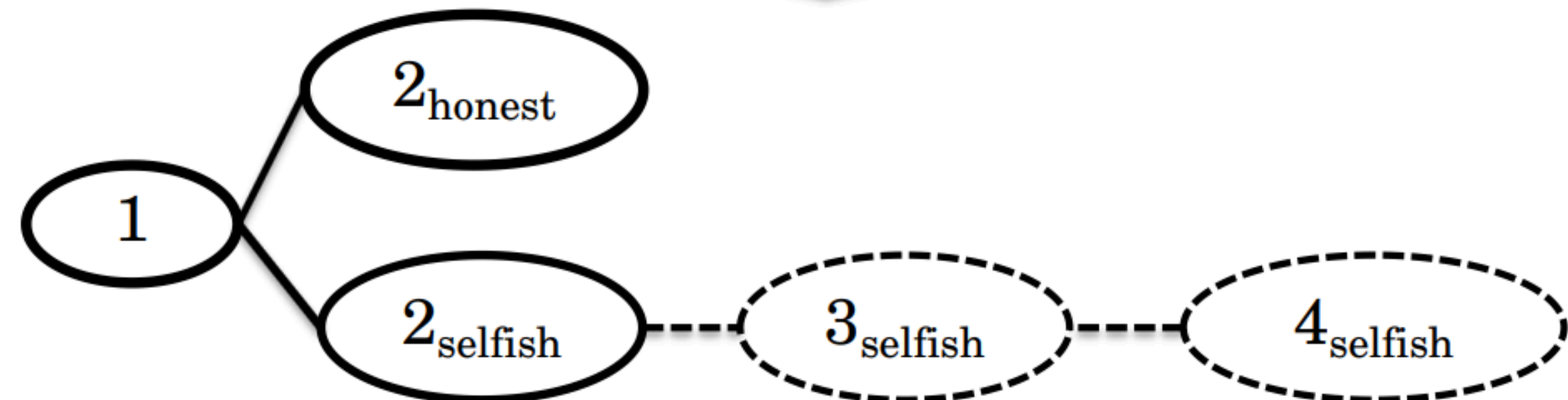
1) Private branch consist of three hidden blocks



2) Honest miners publish a block



3) Selfish miner reveals one of their blocks so there are two competing public branches



# Consensus Models

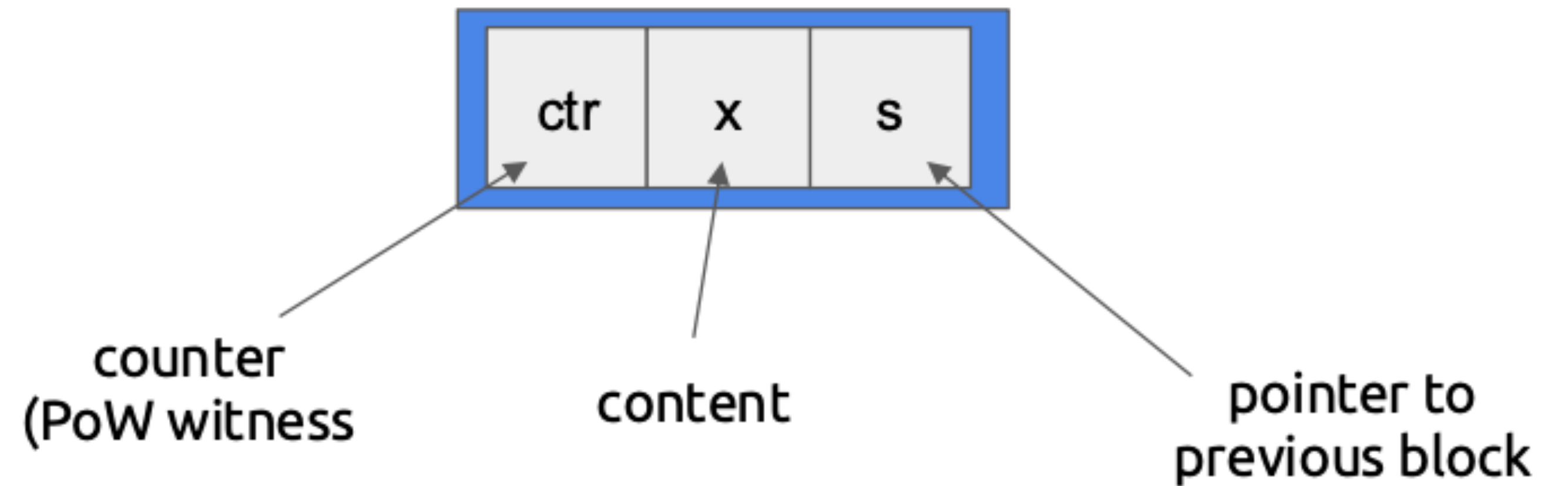
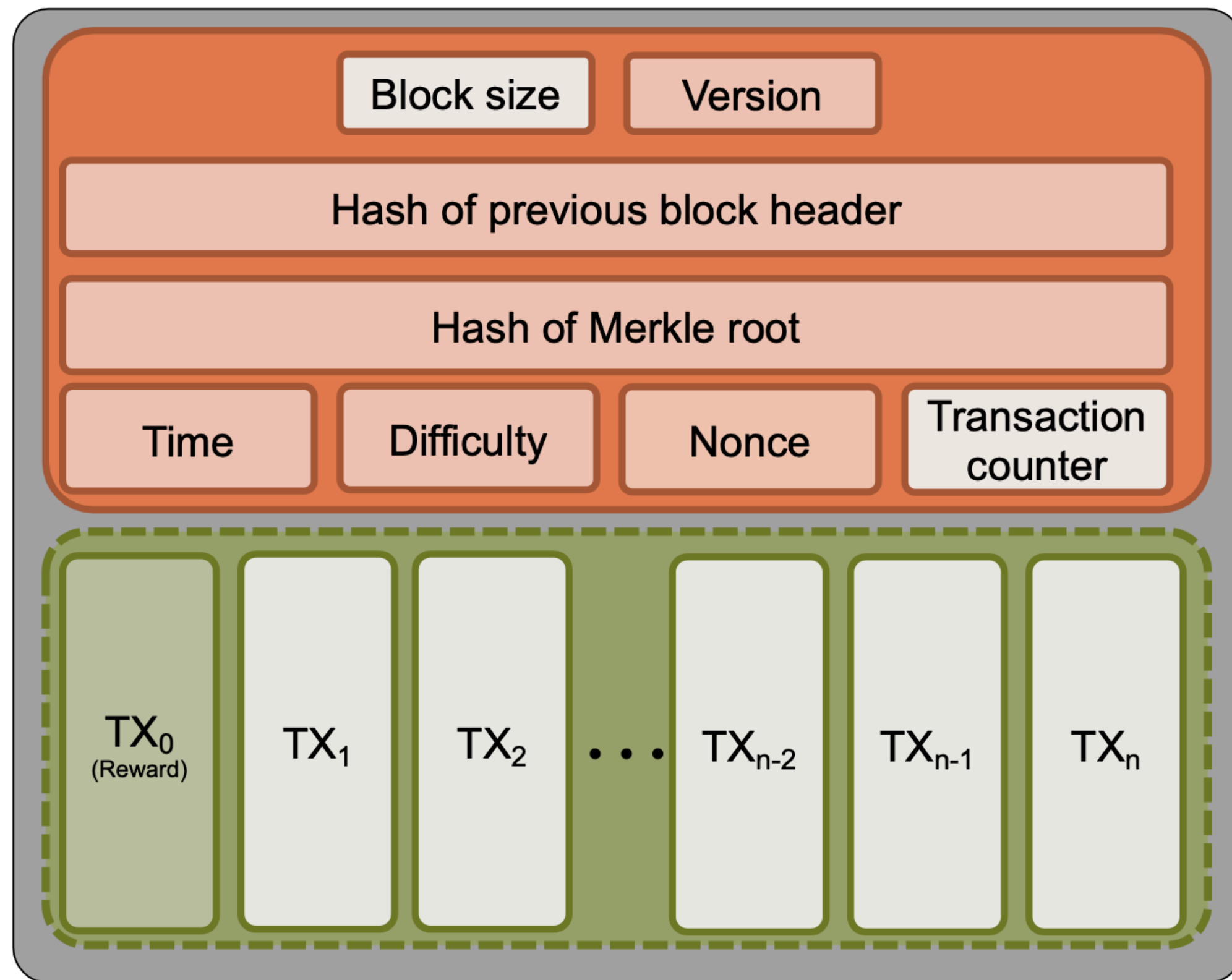
Consistency  $\leftarrow$  *(strong) Common Prefix*

- need to exclude  $k$  most recent blocks

Liveness  $\leftarrow$  *Chain Growth and Chain Quality*

- leave sufficient time for chain to grow
- apply chain quality to ensure that at least one honest block is included

# Consensus Models: Bitcoin



# Consensus Models: PoW

---

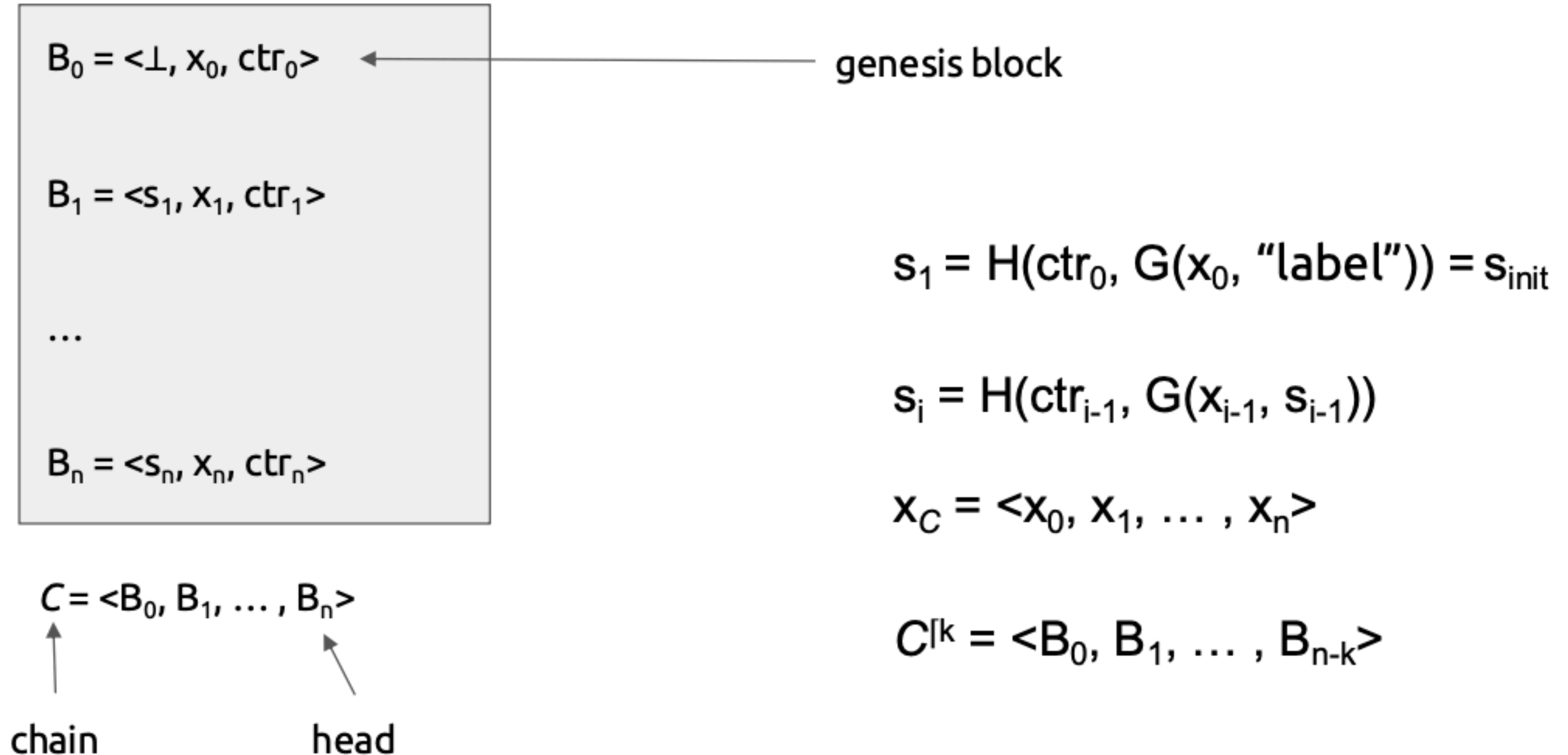
**Algorithm 3** The *proof of work* function, parameterized by  $q$ ,  $T$  and hash functions  $H(\cdot)$ ,  $G(\cdot)$ .  
The input is  $(x, \mathcal{C})$ .

---

```
1: function pow( $x, \mathcal{C}$ )
2:   if  $\mathcal{C} = \varepsilon$  then                                     ▷ Determine proof of work instance
3:      $s \leftarrow 0$ 
4:   else
5:      $\langle s', x', ctr' \rangle \leftarrow \text{head}(\mathcal{C})$ 
6:      $s \leftarrow H(ctr', G(s', x'))$ 
7:   end if
8:    $ctr \leftarrow 1$ 
9:    $B \leftarrow \varepsilon$ 
10:   $h \leftarrow G(s, x)$ 
11:  while ( $ctr \leq q$ ) do
12:    if ( $H(ctr, h) < T$ ) then                               ▷ This  $H(\cdot)$  invocation subject to the  $q$ -bound
13:       $B \leftarrow \langle s, x, ctr \rangle$ 
14:      break
15:    end if
16:     $ctr \leftarrow ctr + 1$ 
17:  end while
18:   $\mathcal{C} \leftarrow \mathcal{C}B$                                          ▷ Extend chain
19:  return  $\mathcal{C}$ 
20: end function
```

---

# Consensus Models: PoW



# Consensus Models: PoW

---

**Algorithm 1** The *chain validation predicate*, parameterized by  $q, T$ , the hash functions  $G(\cdot), H(\cdot)$ , and the *content validation predicate*  $V(\cdot)$ . The input is  $\mathcal{C}$ .

---

```
1: function validate( $\mathcal{C}$ )
2:    $b \leftarrow V(\mathbf{x}_{\mathcal{C}})$ 
3:   if  $b \wedge (\mathcal{C} \neq \varepsilon)$  then                                      $\triangleright$  The chain is non-empty and meaningful w.r.t.  $V(\cdot)$ 
4:      $\langle s, x, ctr \rangle \leftarrow \text{head}(\mathcal{C})$ 
5:      $s' \leftarrow H(ctr, G(s, x))$ 
6:     repeat
7:        $\langle s, x, ctr \rangle \leftarrow \text{head}(\mathcal{C})$ 
8:       if  $\text{validblock}_q^T(\langle s, x, ctr \rangle) \wedge (H(ctr, G(s, x)) = s')$  then
9:          $s' \leftarrow s$                                               $\triangleright$  Retain hash value
10:         $\mathcal{C} \leftarrow \mathcal{C}^{\lceil 1}$                                       $\triangleright$  Remove the head from  $\mathcal{C}$ 
11:       else
12:          $b \leftarrow \text{False}$ 
13:       end if
14:     until  $(\mathcal{C} = \varepsilon) \vee (b = \text{False})$ 
15:   end if
16:   return ( $b$ )
17: end function
```

---

# Consensus Models: PoW

---

**Algorithm 2** The function that finds the “best” chain, parameterized by function  $\max(\cdot)$ . The input is  $\{C_1, \dots, C_k\}$ .

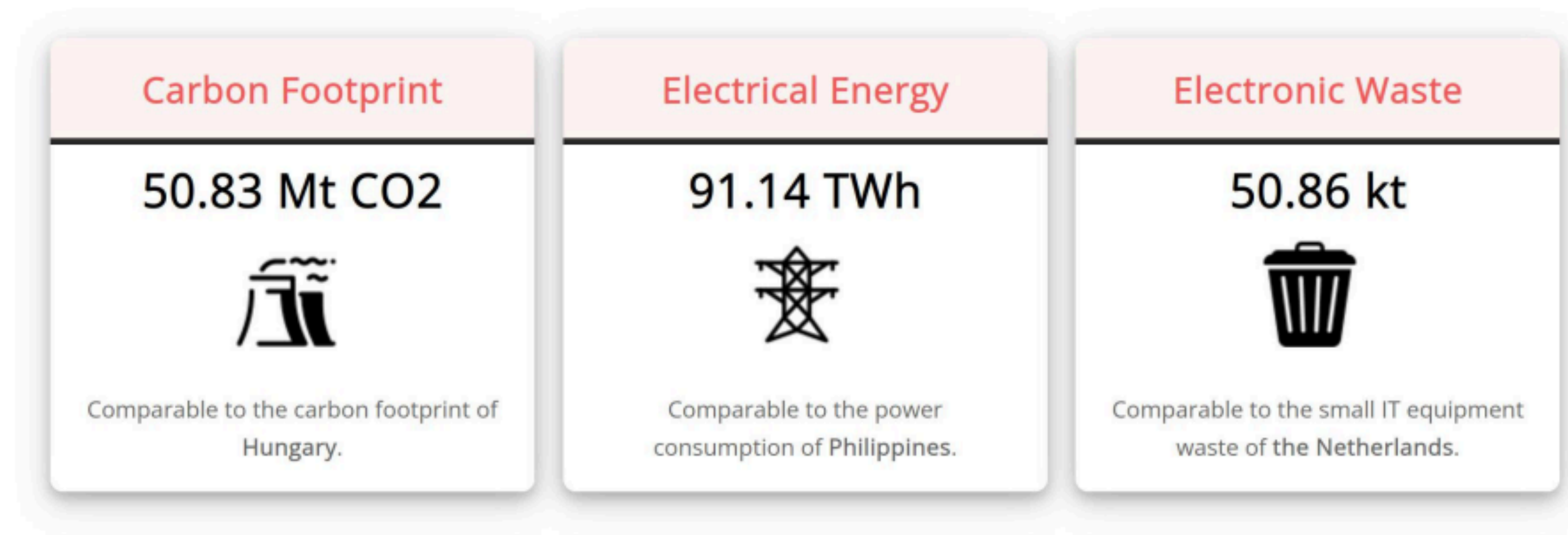
---

```
1: function maxvalid( $C_1, \dots, C_k$ )
2:    $temp \leftarrow \varepsilon$ 
3:   for  $i = 1$  to  $k$  do
4:     if validate( $C_i$ ) then
5:        $temp \leftarrow \max(C_i, temp)$ 
6:     end if
7:   end for
8:   return  $temp$ 
9: end function
```

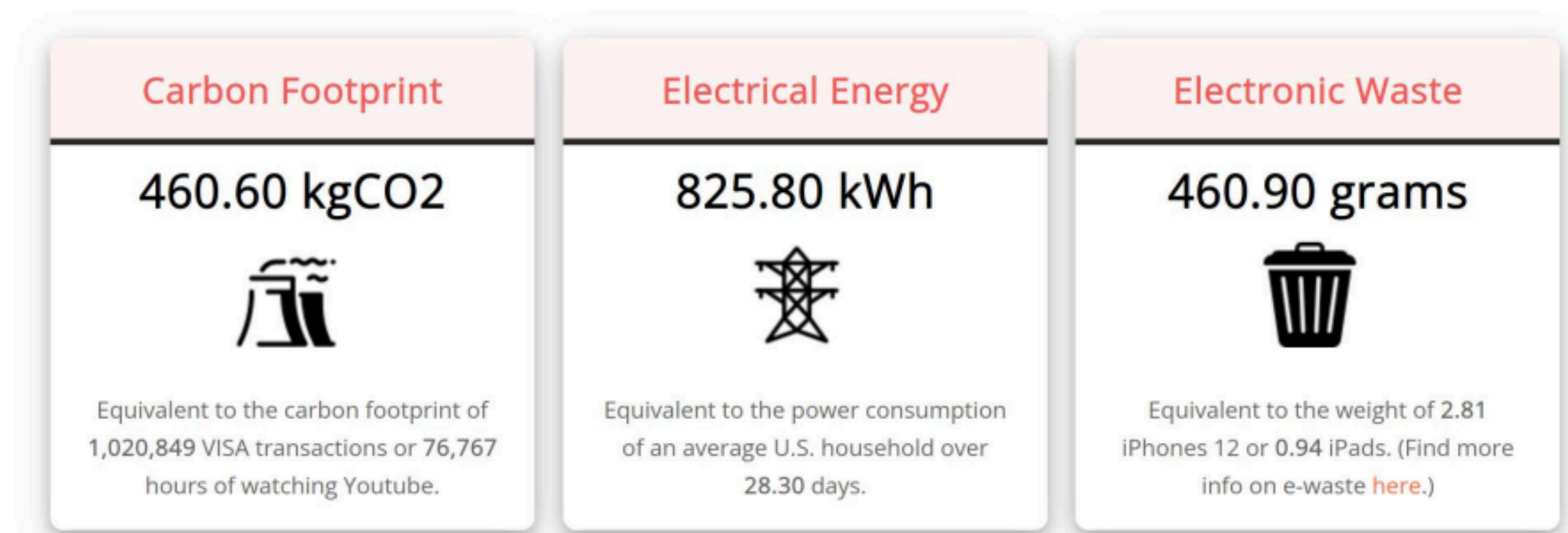
---

# Proof-of-Work (PoW): Environmental Impact

## Annualized Total Bitcoin Footprints



## Single Bitcoin Transaction Footprints



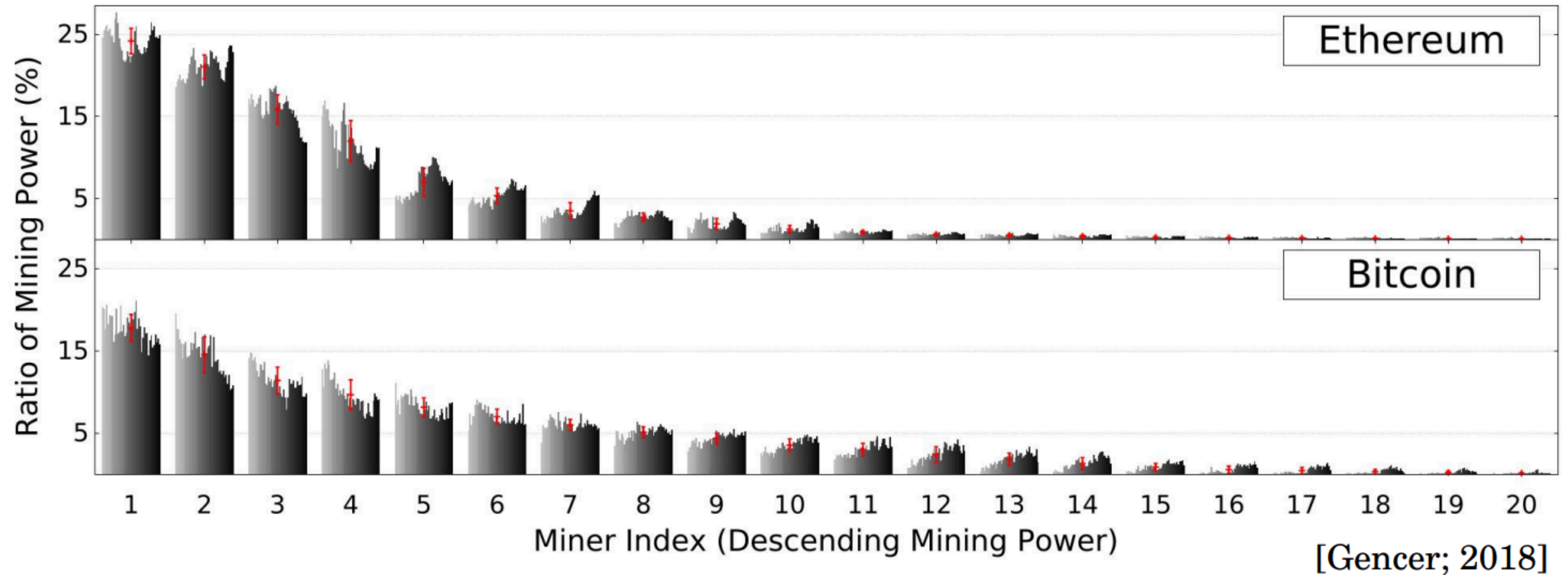
# Proof-of-Work (PoW): Environmental Impact

**High energy consumption:** As a motivating example, consider Bitcoin, which has a current hash rate of  $\approx 300$  exahashes (exa =  $10^{18}$ ) per second, i.e. the hash rate of Bitcoin is of the order of  $10^{20}$  hashes per second around the world across all miners (Figure 1). The total energy consumption for computing so many hashes more than that of Argentina, a country with over 45 million residents. While a significant percentage of that energy comes from renewable sources, it is still a significant consumer of energy.

# Proof-of-Work (PoW): Information Asymmetry

- Nodes that see blocks earlier have an advantage
  - Can start mining on the most recent block before others
- Nodes that do not see blocks in time have a disadvantage
  - Will mine on an outdated version of the chain
- Nodes can intentionally hide blocks
  - Selfish Mining

# Proof-of-Work (PoW): Centralization



## In 2017

- Bitcoin: over 50% of mining power controlled **by four miners**
- Ethereum: over 50% of mining power controlled **by three miners**

# Proof-of-Work (PoW): Centralization

## **Reasons:**

- More efficient to operate mining pools at large scale
  - Some fixed cost, e.g., cooling, easier to amortize
- Large mining pools have a more reliable revenue stream
  - Small miners may not find blocks for a long time
- Miners see their own blocks first
  - More likely that their next block will be part of the winning chain

# Proof-of-Work (PoW)

Bitcoin's Proof of Work can be parallelized

Parties tend to form mining *pools*

- Instead of working separately, work **together** to solve PoW for the same block.
- By collecting “**shares**” (small hashes of the block that are not quite as small as needed) one can prove how much they contributed.

# Proof-of-X

## Alternatives to Proof-of-Work?

Proof-of-X refers to cryptographic protocols designed to replicate the important features of proof-of-work using resource  $X$ , without the aforementioned drawbacks. Most recently released blockchains use **Proof-of-Stake (PoS)**, where *one-CPU-one-vote* of a PoW protocol is replaced with *one-coin-one-vote* in the Proof-of-Stake protocol. PoS protocols strive to be resistant to Sybil Attacks by forcing an attacker to acquire a large portion of the blockchain's tokens.

# Proof-of-Stake (PoS)

**Idea:** Assign voting power by stake, not mining power

- Stake is the amount of currency held by a particular entity

**Challenge 1:** How to pick block creators?

- We need some kind of randomness
- True randomness is hard to generate in the blockchain setting
- Attackers might try to influence the random number generation (*grinding attack*)

**Challenge 2:** *Nothing at Stake*

- Block creation is computationally cheap
- Easy for an attacker to try to create many blocks

# Proof-of-Stake (PoS)

Support **varying voting power**

- Either total balance of an entity or staked balance
- Staked balance: Need to lock up some money to be used for staking
  - Simpler to implement but less flexible

Support **delegation**

- Not everyone might have the resources to participate in consensus
- Allow for “stake pools”

# Proof-of-Stake (PoS)

## **Randomize Schedule**, e.g, Ouroboros

- Time is split into fixed-size slots
- Set a sequence of block creators in advance, each responsible for one slot

## **Random Selection**, e.g., Algorand

- Time is split into fixed-size slots
- Every node has some chance to be part of the committee of a block

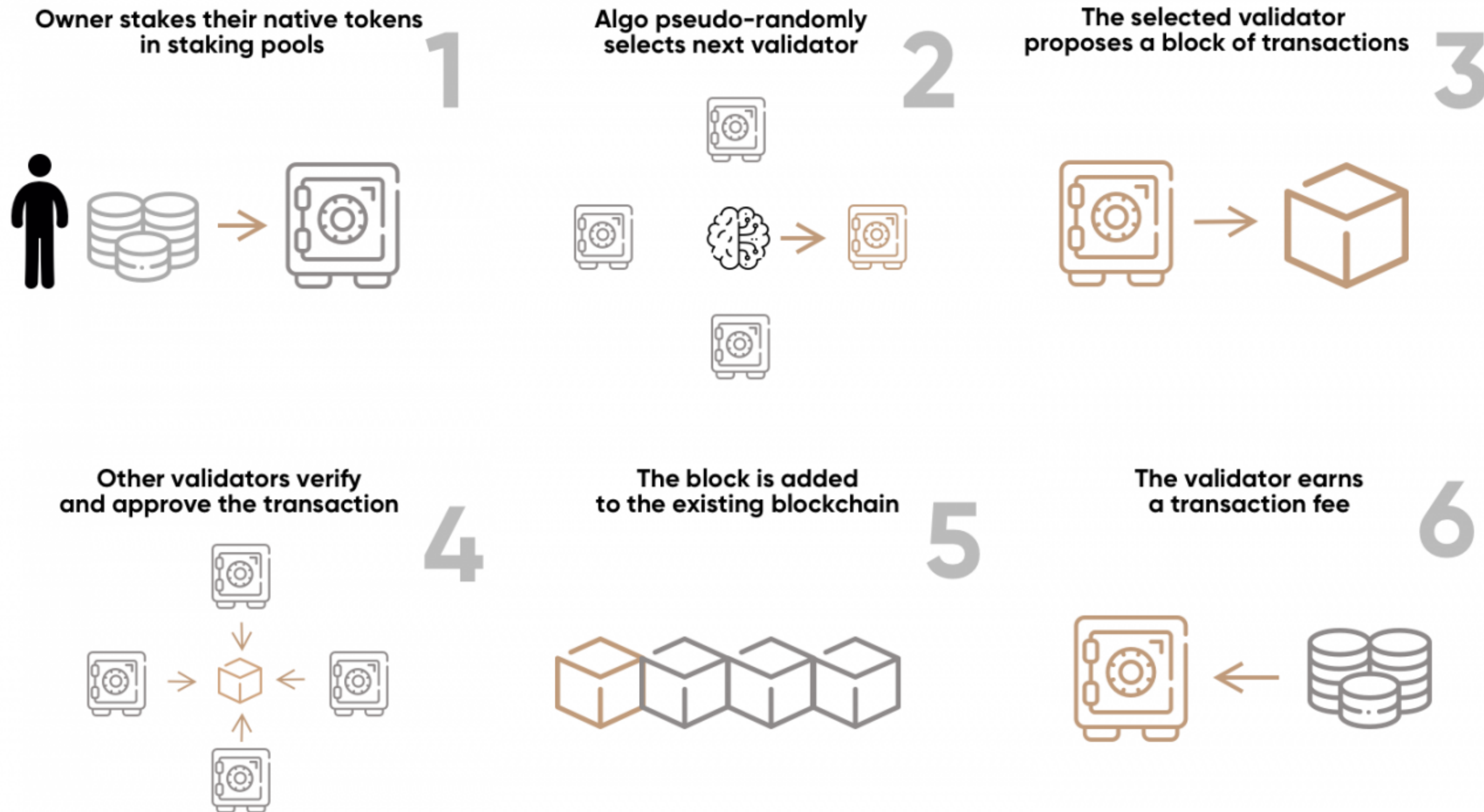
## **Random Sampling**, e.g., Avalanche

- Ask other nodes about which transaction they have accepted
- Eventually converge to the same set of accepted transactions
- More about this in another lecture

**Always:** Voting power (or chance to be selected) is proportional to stake

# Proof-of-Stake (PoS)

## HOW STAKING WORKS IN THE PROOF-OF-STAKE CONSENSUS MECHANISM



Source: SEBA Research

# Proof-of-Stake (PoS)

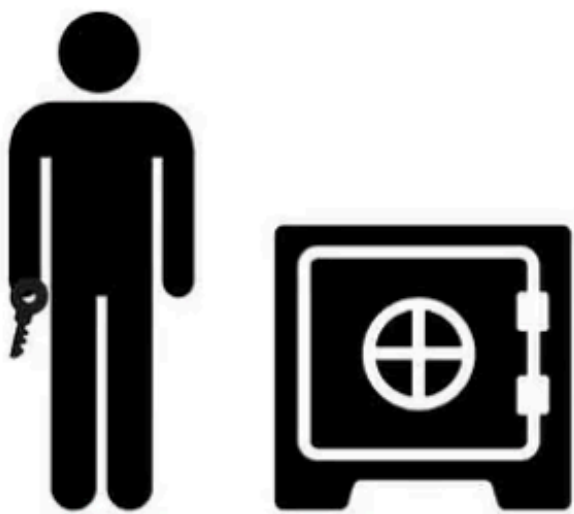
## PROOF-OF-WORK

OR

## PROOF-OF-STAKE



THE PROBABILITY OF MINING A BLOCK IS DEPENDENT ON HOW MUCH WORK IS DONE BY THE MINER



PERSON CAN "MINE" DEPENDING ON HOW MANY COINS THEY HOLD



PAYOUTS BECOMES SMALLER AND SMALLER FOR BITCOIN MINERS, THERE IS LESS INCENTIVE TO AVOID A 51% ATTACK



THE POS SYSTEMS MAKES ANY 51% ATTACK MORE EXPENSIVE



POW SYSTEMS HAVE POWERFUL MINING COMMUNITIES-BUT TEND TO BECOME CENTRALIZED OVER TIME



POS SYSTEMS ARE MORE DECENTRALIZED-BUT MUST WORK HARD TO BUILD COMMUNITIES AROUND THEIR COINS

# Proof-of-Stake (PoS)

**Fast Confirmation:** Bitcoin requires a long confirmation time for transactions because one needs to wait for a block to be buried under  $k$  blocks (this process can take hours), whereas some Proof-of-Stake protocols can confirm transactions in seconds.

**Accountability:** While Nakamoto's paper introduced the use of incentives in consensus, there is no punishment system for malicious actions since miners have no identity. On the other hand, in a one-coin-one-vote Proof-of-Stake system, we know who is participating in the system, so it is **possible** to take away a staker's stake if they violated the protocol.

# Proof-of-Stake (PoS): Attacks

**Prediction Attack:** In Proof-of-Work, it is completely unpredictable who will get the next block. However, in Proof-of-Stake, this is no longer the case. We will see in this lecture how Proof-of-Stake leads to predictability. Predictability leads to attacks where the adversary can bribe the winner of the next block to carry out malicious actions. We will also see in the next lecture how cryptography can be used to ameliorate this problem to some extent.

**History Revision Attacks:** In Proof-of-Work, it is impossible to rewrite the history of the blockchain over a long period of time since one cannot redo all the work done in a long period of time, a property of the arrow of time. However, since the concept of “work” does not exist in Proof-of-Stake, it is easier to rewrite history, thus opening up the possibility of such attacks.

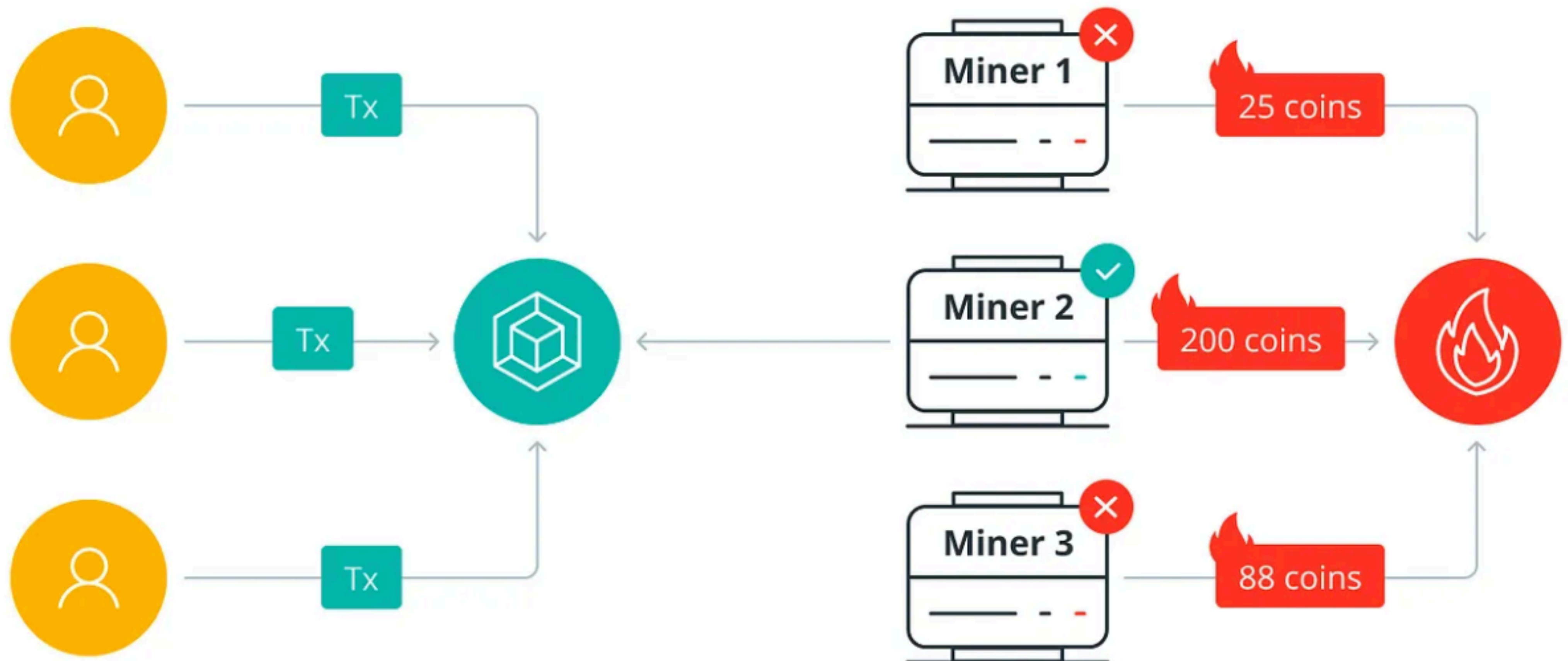
# Proof-of-Burn (PoB)

Basically, for Proof of Burn to work, miners will have to send their coins into the '*eater address*.' This is a publicly verifiable address to all the participants in the network but is not accessible to anyone.

Burn addresses are randomly generated and do not have private keys. That means they are 'black hole' addresses where coins sent to them become permanently irretrievable. Therefore, there's a continual reduction of coins in the system, leading to an increase in the value of the assets.

# Proof-of-Burn (PoB)

## Proof-of-burn



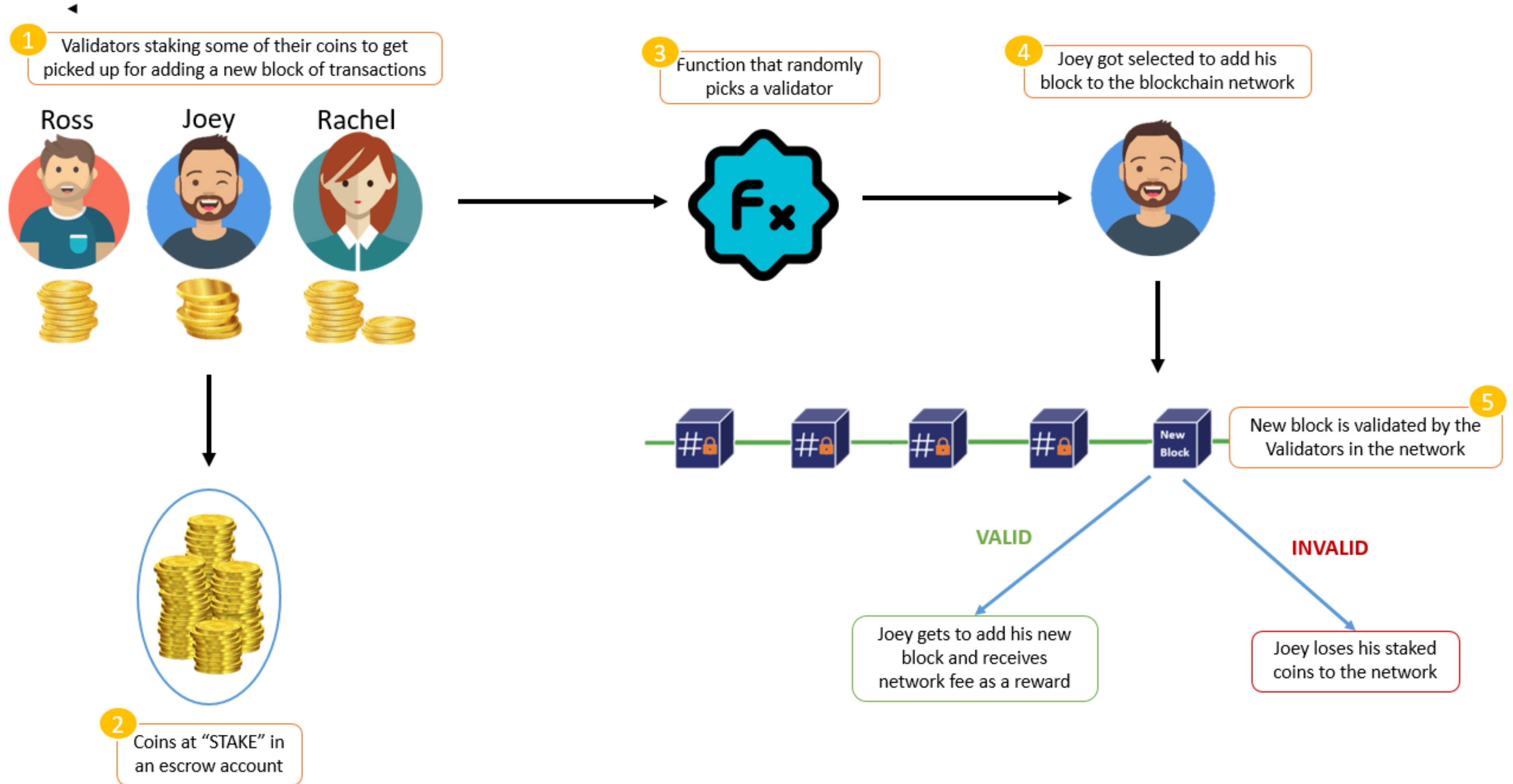
# Delegated Proof-of-Stake (DPoS)

In Delegated Proof of Stake (DPoS) mechanism, the **validators' or delegates'** selection is based on voting. It's an **additional layer to PoS protocol**. It works similarly to the real-life election and voting system.

DPoS was coined in 2014 by an American cryptocurrency entrepreneur and software developer, **Daniel Larimer**. The DPoS maintains an election process where a selected number of delegates (or validators) are chosen. Those delegates are responsible and trusted by the network participants for validating each new block added to the network.

Delegated Proof of Stake (DPoS) is a more democratic version of the Proof of Stake (PoS) mechanism. Both algorithms are used as an alternative to high energy-consuming PoW by Bitcoin.

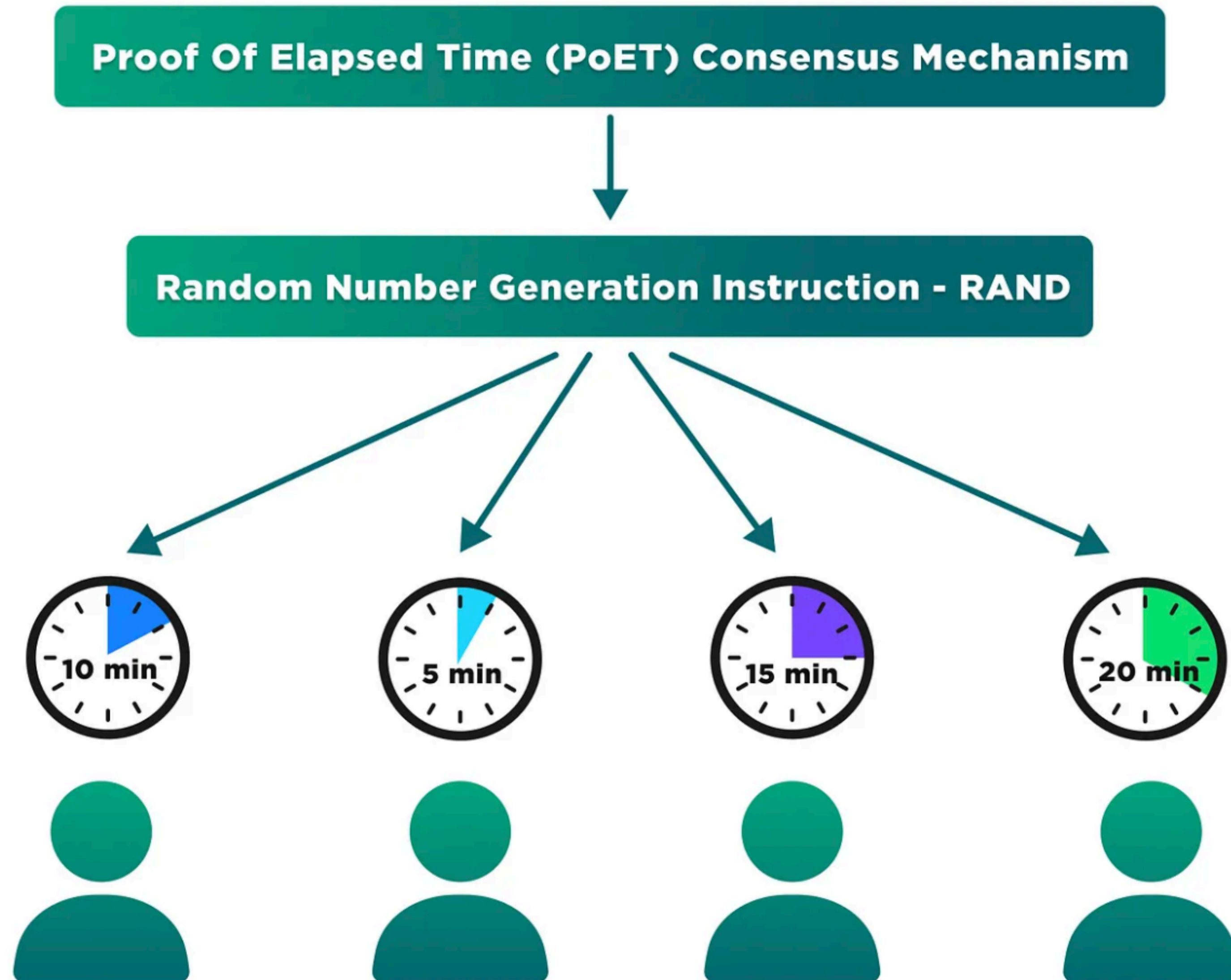
# Delegated Proof-of-Stake (DPoS)



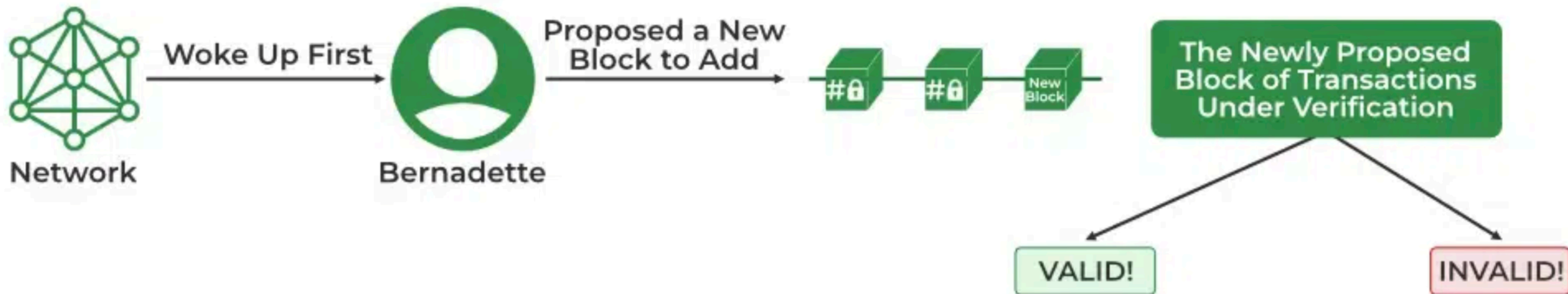
# Proof-of-Elapsed-Time (PoET)

- Assign an amount of waiting time to each node in the network randomly.
- The node must sleep or do another task during the wait time.
- The node that gets the shortest waiting time wakes up the first and is allowed to produce the block.
- The node transmits the information to other network participants after completing the block production.

# Proof-of-Elapsed-Time (PoET)



# Proof-of-Elapsed-Time (PoET)



# Graph-based Blockchain

## **Idea**

- Remove notion of blocks entirely
- Network directly agrees on a directed acyclic graph of transactions

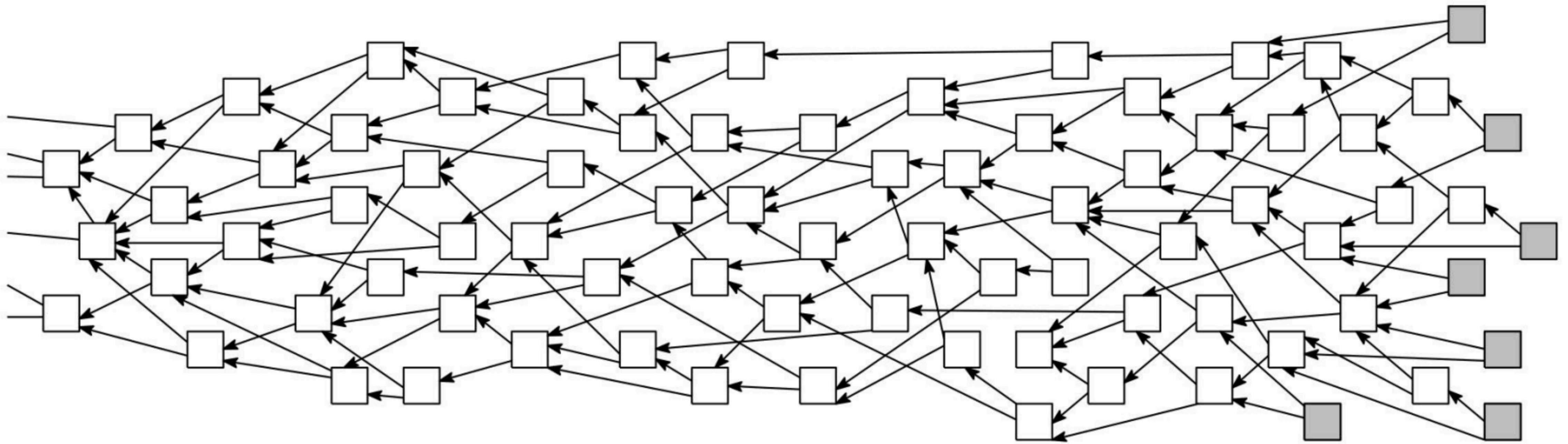
## **Works well with the UTXO model**

- For an honest client, a transaction never conflicts with another
- An attacker might still issue conflicting “rogue” transactions

## **Problem**

- When is a transaction confirmed?

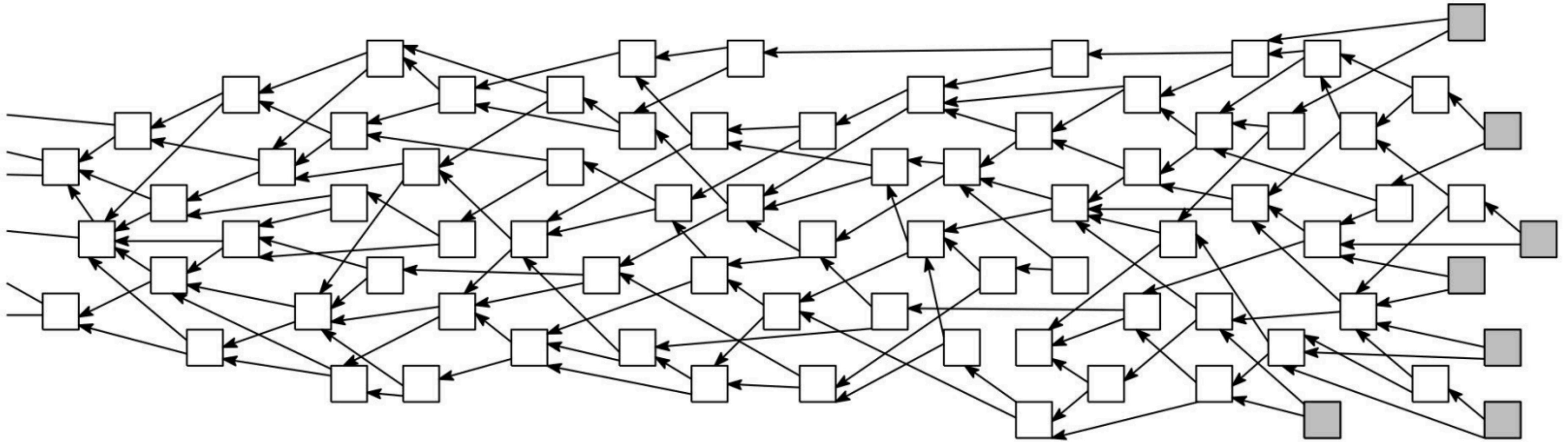
# Graph-based Blockchain: IOTA



Anyone can issue a transaction

- Transactions need to include a proof of work to reduce spam
- Each transaction references at least two predecessors and confirms them

# Graph-based Blockchain: IOTA



What could go wrong?

- Attacker's confirm their own transaction by creating other transactions
- The DAG could diverge into multiple conflicting branches

# Graph-based Blockchain: IOTA

IOTA relies on a centralized entity to frequently checkpoint the network

- Creates *milestones* (empty transactions) in frequent intervals that confirms all valid transactions

This entity can stop the network from finalizing transactions if it does not issue new milestones

[Home](#) / [Tech](#) / [Security](#)

## **IOTA cryptocurrency shuts down entire network after wallet hack**

Hackers exploit vulnerability in official IOTA wallet to steal millions

# Avalanche

## **An new type of consensus protocols**

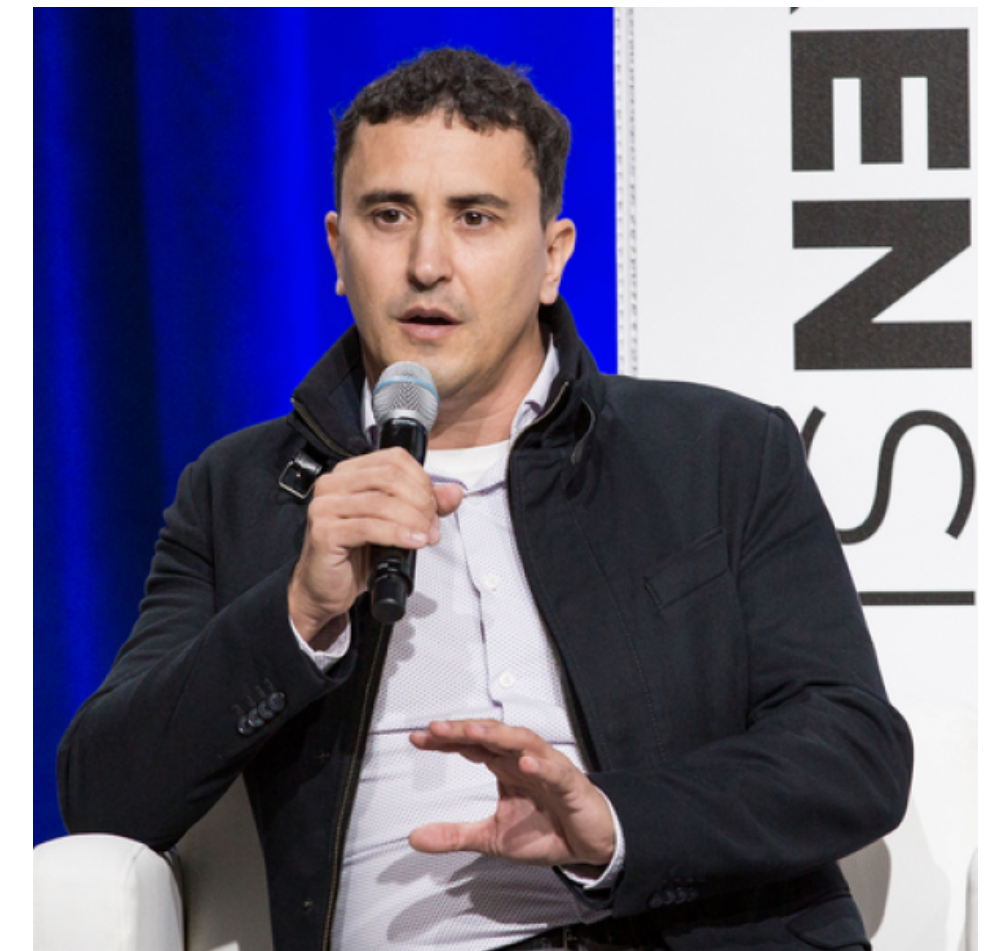
- Relies on probabilistic sampling

## **Similar properties as Nakamoto consensus**

- Works with very large networks
- Can handle changing membership
- Constant communication complexity per node and round
  - Larger networks need more communication rounds

## **Differences to Nakamoto consensus**

- Requires knowledge of most nodes in the network
- Can work with, both, DAGs and chains
- Fast confirmation times



**Emin Gün Sirer**

# Avalanche: Slush

Simplified version of Avalanche

- No Byzantine fault tolerance
- Binary decision (either “Red” or “Blue”)

Some nodes start with a specific color

- Others adopt a color when first queried

Each round, nodes sample a random set of  $k$  other nodes

- If a majority of sampled peers responds with the same color, adopt the color
- Majority is defined by parameter  $\alpha$

Execut for  $m$  rounds

- Protocol will reach agreement with a high enough value of high  $m$
- $m$  grows logarithmically with the network size

```
1: procedure ONQUERY( $v, col'$ )
2:   if  $col = \perp$  then  $col := col'$ 
3:   RESPOND( $v, col$ )
4: procedure SLUSHLOOP( $u, col_0 \in \{R, B, \perp\}$ )
5:    $col := col_0$  // initialize with a color
6:   for  $r \in \{1 \dots m\}$  do
7:     // if  $\perp$ , skip until ONQUERY sets the color
8:     if  $col = \perp$  then continue
9:     // randomly sample from the known nodes
10:     $\mathcal{K} := \text{SAMPLE}(\mathcal{N} \setminus u, k)$ 
11:     $P := [\text{QUERY}(v, col) \quad \textbf{for } v \in \mathcal{K}]$ 
12:    for  $col' \in \{R, B\}$  do
13:      if  $P.\text{COUNT}(col') \geq \alpha$  then
14:         $col := col'$ 
15:  ACCEPT( $col$ )
```

# Avalanche: Snowflake

Slush with Byzantine Fault-Tolerance

- Binary decision (either “Red” or “Blue”)

Some nodes start with a specific color

- Others adopt a color when first queried

Each round, nodes sample a random set of  $k$  other nodes

- If a majority of sampled peers respond with the same color, adopt that color
- If we already have adopted the color, increase counter  $cnt$
- If we have not adopted the color yet, reset the counter

Run until confidence exceeds some threshold  $\beta$

```
1: procedure SNOWFLAKELOOP( $u, col_0 \in \{R, B, \perp\}$ )
2:    $col := col_0, cnt := 0$ 
3:   while undecided do
4:     if  $col = \perp$  then continue
5:      $\mathcal{K} := \text{SAMPLE}(\mathcal{N} \setminus u, k)$ 
6:      $P := [\text{QUERY}(v, col) \quad \textbf{for } v \in \mathcal{K}]$ 
7:      $maj := \text{false}$ 
8:     for  $col' \in \{R, B\}$  do
9:       if  $P.\text{COUNT}(col') \geq \alpha$  then
10:         $maj := \text{true}$ 
11:        if  $col' \neq col$  then
12:           $col := col', cnt := 1$ 
13:        else  $cnt++$ 
14:        if  $cnt \geq \beta$  then ACCEPT( $col'$ )
15:   if  $maj = \text{false}$  then  $cnt := 0$ 
```

# Avalanche: Snowball

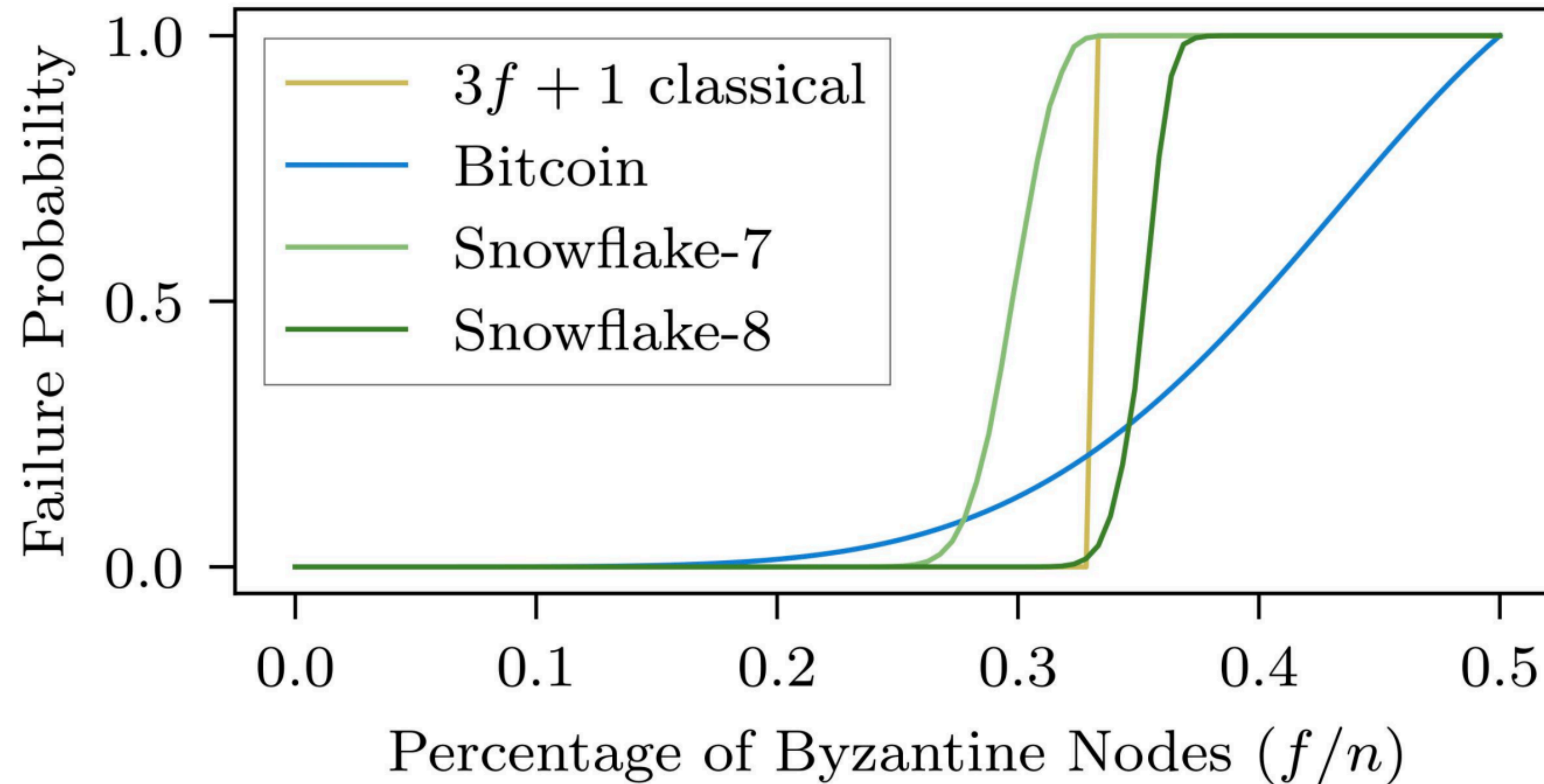
Like Snowflake, but

- Keeps track of majorities reached per color as *confidence value*  $d$
- Confidence is increased every time a majority is reached in a voting round
- Only change decision when confidence for one color exceeds confidence for the other color

More resistant against Byzantine actors

```
1: procedure SNOWBALLLOOP( $u, col_0 \in \{R, B, \perp\}$ )
2:    $col := col_0, lastcol := col_0, cnt := 0$ 
3:    $d[R] := 0, d[B] := 0$ 
4:   while undecided do
5:     if  $col = \perp$  then continue
6:      $\mathcal{K} := \text{SAMPLE}(\mathcal{N} \setminus u, k)$ 
7:      $P := [\text{QUERY}(v, col) \text{ for } v \in \mathcal{K}]$ 
8:      $maj := \text{false}$ 
9:     for  $col' \in \{R, B\}$  do
10:      if  $P.\text{COUNT}(col') \geq \alpha$  then
11:         $maj := \text{true}$ 
12:         $d[col']++$ 
13:        if  $d[col'] > d[col]$  then
14:           $col := col'$ 
15:        if  $col' \neq lastcol$  then
16:           $lastcol := col', cnt := 1$ 
17:        else  $cnt++$ 
18:        if  $cnt \geq \beta$  then ACCEPT( $col'$ )
19:     if  $maj = \text{false}$  then  $cnt := 0$ 
```

# Avalanche: Fault-Tolerance



With  $k=10$  and  $\beta=250$  for Avalanche

For Bitcoin, it models the probability that a block with 6 confirmations (1 hour) will be reorganized